SVEUČILIŠTE U SPLITU

SVEUČILIŠNI ODJEL ZA STRUČNE STUDIJE

Stručni prijediplomski studij Računarstvo

TIN ŠEGVIĆ

Z A V R Š N I R A D

SUSTAV ZA UDALJENI NADZOR LINUX SUSTAVA

Split, rujan 2026.

SVEUČILIŠTE U SPLITU

SVEUČILIŠNI ODJEL ZA STRUČNE STUDIJE

Stručni prijediplomski studij Računarstvo

Predmet: Odabrani alati i naredbe u Linuxu

Z A V R Š N I R A D

Kandidat: Tin Šegvić

Naslov rada: Sustav za udaljeni nadzor Linux sustava

Mentor: Nikola Grgić, v. pred.

Split, rujan 2026.

Sadržaj

Sažetak **1**

1. Uvod **3**

2. Korištene tehnologije i alati **4**

2.1. Programski jezik Python i biblioteka psutil ... 4

2.2. Razvojni okvir Flask i sučelje REST ... 4

2.3. Baza podataka SQLite ... 4

2.4. Tehnologije korisničkog sučelja ... 5

2.5. Sustav za upravljanje verzijama Git ... 5

2.6. Razvojno okruženje ... 5

3. Arhitektura sustava **6**

3.1. Pregled arhitekture ... 6

3.2. Sučelje REST ... 7

3.3. Model baze podataka ... 7

3.4. Tijek podataka kroz sustav ... 9

3.5. Nefunkcionalni zahtjevi i ograničenja ... 10

3.6. Format poruka između agenta i poslužitelja ... 11

4. Implementacija sustava **13**

4.1. Agentska aplikacija ... 13

4.2. Poslužiteljska aplikacija ... 15

4.2.1. Provjera i normalizacija ulaznih podataka ... 17

4.3. Sustav upozorenja ... 18

4.4. Udaljeno izvršavanje naredbi ... 20

4.5. Sigurnosni mehanizmi ... 21

4.6. Dohvat povijesnih podataka ... 21

4.7. Održavanje baze podataka 22

5. Korisničko sučelje 24

5.1. Prijava korisnika i nadzorna ploča 24

5.2. Detaljni prikaz nadziranog računala 25

5.3. Upravljanje procesima 26

5.4. Upozorenja i postavke pragova 26

5.5. Upravljanje nadziranim računalima 27

6. Ispitivanje sustava 29

6.1. Ispitno okruženje 29

6.2. Provjera točnosti prikupljenih podataka 29

6.3. Ispitivanje sustava upozorenja 29

6.4. Ispitivanje stabilnosti komunikacije 30

6.5. Ispitivanje udaljenog izvršavanja naredbi 30

6.6. Pregled rezultata ispitivanja 30

6.7. Automatizirano ispitivanje 31

6.8. Ograničenja provedenog ispitivanja 33

7. Zaključak 34

Literatura 35

Popis slika 36

Popis tablica 37

Popis ispisa 37

Dodatak 38

Sažetak

U radu su opisani projektiranje i implementacija sustava za udaljeni nadzor računala s operativnim sustavom Linux. Sustav se sastoji od dviju komponenti. Agentska aplikacija koja se izvodi na svakom nadziranom računalu te u pravilnim vremenskim intervalima prikuplja podatke o opterećenju procesora, zauzeću radne memorije i diska, mrežnom prometu, broju aktivnih procesa i vremenu rada sustava.

Prikupljeni podatci šalju se središnjem poslužitelju postavljenom za potrebe ovoga rada. Poslužiteljska aplikacija zaprima podatke putem sučelja REST, pohranjuje ih u bazu podataka SQLite te ih prikazuje korisniku u web aplikaciji. Uz prikaz podataka implementiran je sustav upozorenja koji obavještava korisnika o prekoračenju definiranih pragova i o nedostupnosti nadziranog računala, kao i podsustav za udaljeno upravljanje procesima na nadziranim računalima.

Sustav je ispitan u kontroliranom okruženju virtualnih strojeva s operativnim sustavom Linux, pri čemu je provjerena točnost prikupljenih podataka, stabilnost komunikacije te ispravnost rada sustava upozorenja.

Ključne riječi: *agent, Linux, nadzor sustava, sučelje REST, web aplikacija*

Summary

Remote Monitoring System for Linux Systems

This thesis describes the design and implementation of a system for remote monitoring of computers running the Linux operating system. The system consists of two components. An agent application runs on every monitored computer and collects data on processor load, memory and disk usage, network traffic, the number of active processes and system uptime at regular time intervals.

The collected data is sent to a central server set up for the purposes of this thesis. The server application receives the data through a REST interface, stores it in an SQLite database, and presents it to the user in a web application. In addition to data presentation, an alerting subsystem notifies the user when the defined thresholds are exceeded or when a monitored

computer becomes unavailable, as well as a subsystem for remote process management on the monitored computers.

The system was tested in a controlled environment of Linux virtual machines, verifying the accuracy of the collected data, the stability of communication, and the correct operation of the alerting subsystem.

Keywords: *agent, Linux, REST interface, system monitoring, web application*

1. Uvod

Održavanje više računalnih sustava zahtijeva stalni uvid u njihovo stanje. Administrator koji ne raspolaže središnjim nadzornim sustavom prisiljen je ručno se spajati na svako računalo i provjeravati podatke koji ga zanimaju (npr. opterećenje procesora, zauzeće radne memorije i diska, mrežni promet, broj aktivnih procesa i vrijeme rada sustava), što je sporo, nepraktično i podložno pogreškama. Problemi poput prekomjernog opterećenja uzrokovanoga pojedinim procesom ili popunjenosti diska često se uočavaju tek kada dovedu do pada ili znatnog usporavanja sustava.

Motivacija za izradu ovog rada jest izgradnja jednostavnog i samostalnog sustava za udaljeni nadzor koji ne zahtijeva složenu infrastrukturu kakvu koriste velika nadzorna rješenja, primjerice sustavi Zabbix ili Nagios. Navedena rješenja pokrivaju širok skup mogućnosti [1], [2], no njihovo postavljanje i održavanje zahtijeva razmjerno velik trud koji u okruženjima s manjim brojem računala često nije opravdan. Sustav opisan u ovom radu pokriva najvažnije potrebe nadzora manjeg broja računala s operativnim sustavom Linux.

Problem je riješen izradom dviju komponenti. Agentska aplikacija napisana je u programskom jeziku Python i izvodi se na svakom računalu koje se nadzire. Agent u definiranim vremenskim intervalima prikuplja sistemske pokazatelje pomoću biblioteke `psutil` i šalje ih središnjem poslužitelju u formatu JSON putem sučelja REST. Poslužiteljska aplikacija izgrađena je pomoću razvojnog okvira Flask i izvodi se na središnjem poslužiteljskom računalu postavljenom za potrebe ovoga rada. Ona zaprima podatke, pohranjuje ih u bazu podataka SQLite te ih prikazuje u web sučelju uz pomoć grafova izrađenih bibliotekom `Chart.js`. Uz prikaz podataka sustav generira upozorenja pri prekoračenju pragova i omogućuje udaljeno upravljanje procesima nadziranih računala.

U drugom poglavlju ukratko su opisane tehnologije i alati korišteni pri izradi rada. Treće poglavlje prikazuje arhitekturu sustava, sučelje REST i model baze podataka. Četvrto poglavlje detaljno opisuje implementaciju agentske i poslužiteljske komponente, sustava upozorenja te udaljenog izvršavanja naredbi. U petom poglavlju prikazano je korisničko sučelje aplikacije, a u šestom postupak i rezultati ispitivanja. Rad završava zaključkom koji uključuje prijedloge mogućih poboljšanja.

2. Korištene tehnologije i alati

U ovom su poglavlju ukratko opisane tehnologije korištene pri izradi rada, s naglaskom na ulogu svake od njih u sustavu. Uobičajene tehnologije opisane su samo u mjeri potrebnoj za razumijevanje ostatka rada.

2.1. Programski jezik Python i biblioteka psutil

Python je interpretirani programski jezik odabran zbog čitljive sintakse, bogatoga standardnog skupa biblioteka i dostupnosti na svim distribucijama operativnog sustava Linux. U radu je korišten za izradu obiju komponenti sustava.

Biblioteka `psutil` omogućuje čitanje podataka o procesima i resursima računala (procesor, memorija, disk i mreža) na način neovisan o platformi [3]. U agentskoj aplikaciji korištena je za prikupljanje svih sistemskih pokazatelja te za dohvat popisa procesa i slanje signala procesima. Za mrežnu komunikaciju agent koristi biblioteku `requests`, a za izvršavanje udaljenih naredbi modul `subprocess` iz standardne biblioteke.

2.2. Razvojni okvir Flask i sučelje REST

Flask je razvojni okvir za izradu web aplikacija u programskom jeziku Python. Odabran je zato što ne nameće strukturu projekta ni dodatne ovisnosti, a pruža sve što je potrebno za ovaj rad: usmjeravanje zahtjeva HTTP, rad sa sesijama, predloške Jinja2 za generiranje stranica HTML te jednostavnu izradu programskog sučelja [4].

Komunikacija između agenta i poslužitelja ostvarena je prema načelima arhitekturnog stila REST (engl. *Representational State Transfer*). Podatci se razmjenjuju u formatu JSON putem protokola HTTP, a svaki se zahtjev agenta autenticira ključem API.

2.3. Baza podataka SQLite

SQLite je relacijska baza podataka koja se ne izvodi kao zaseban poslužiteljski proces, nego kao biblioteka ugrađena u aplikaciju, pri čemu se svi podatci pohranjuju u jednu datoteku [5]. Time se pojednostavnjuju postavljanje i prijenos sustava, što odgovara

predviđenoj namjeni nadzora manjega broja računala. Pristup bazi ostvaren je modulom `sqlite3` iz standardne biblioteke jezika Python.

2.4. Tehnologije korisničkog sučelja

Korisničko sučelje izrađeno je standardnim web tehnologijama: jezikom HTML za strukturu stranica, jezikom CSS za oblikovanje te jezikom JavaScript za dinamičko osvježavanje prikaza bez ponovnog učitavanja stranice. Za crtanje grafova vremenskih nizova pokazatelja korištena je biblioteka `Chart.js`, koja omogućuje jednostavnu izradu interaktivnih linijskih grafova u elementu `canvas` [6].

2.5. Sustav za upravljanje verzijama Git

Tijekom izrade rada korišten je sustav za upravljanje verzijama Git. Sav je programski kôd pohranjen u udaljenom repozitoriju, čime je zabilježena cjelokupna povijest razvoja projekta i omogućen povrat na prethodna stanja programskog kôda.

2.6. Razvojno okruženje

Razvoj sustava proveden je u okruženju virtualnih strojeva s operativnim sustavom Linux, međusobno povezanih lokalnom mrežom. Takvo okruženje omogućuje istodobno pokretanje poslužiteljske aplikacije i više agenata bez potrebe za dodatnom fizičkom opremom te vjerno odgovara okruženju u kojem bi sustav bio primijenjen. Sastav okruženja korištenoga za ispitivanje sustava opisan je u poglavlju 6. Ispitivanje sustava.

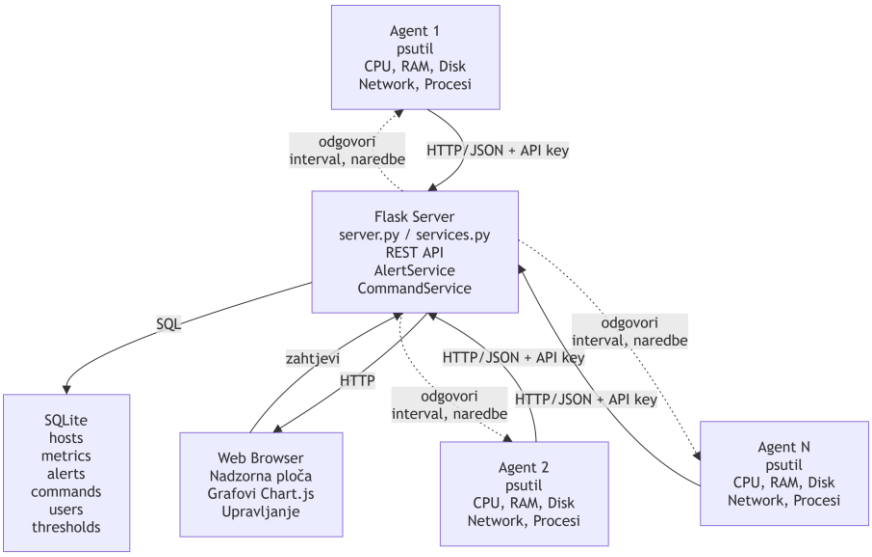
3. Arhitektura sustava

3.1. Pregled arhitekture

Sustav je organiziran prema modelu agent – središnji poslužitelj. Na svakom nadziranom računalu izvodi se agentska aplikacija koja periodično prikuplja pokazatelje i šalje ih poslužitelju. Svu komunikaciju započinje agent prema poslužitelju, čime se pojednostavnjuje mrežna konfiguracija: nadzirana računala ne moraju imati otvorene dolazne priključke, nego je dovoljno da mogu uspostaviti vezu prema poslužitelju.

Poslužiteljska aplikacija obavlja tri zadaće: zaprima i pohranjuje pokazatelje, provjerava prekoračenja pragova i generira upozorenja te poslužuje web sučelje za pregled podataka i upravljanje sustavom. Odgovor poslužitelja na svaku poruku s pokazateljima ujedno prenosi agentu trenutačno važeći interval slanja i eventualne naredbe na čekanju, pa isti komunikacijski kanal služi i za udaljeno upravljanje.

Opisana arhitektura sustava, s prikazom obiju komponenti i smjera komunikacije među njima, prikazana je na slici 1.



Slika 1: Arhitektura sustava

3.2. Sučelje REST

Programsko sučelje poslužitelja podijeljeno je na dio namijenjen agentima i dio namijenjen web sučelju. Agentski je dio dostupan uz autentikaciju ključem API koji se šalje u zaglavlju `X-API-Key`, dok dio namijenjen web sučelju zahtijeva prijavljenoga korisnika. Pregled najvažnijih krajnjih točaka nalazi se u tablici 1.

Tablica 1: Najvažnije krajnje točke sučelja REST

Metoda	Putanja	Opis
POST	`/api/register`	Registracija novog nadziranog računala; poslužitelj vraća jedinstveni ključ API.
POST	`/api/metrics`	Slanje prikupljenih pokazatelja; odgovor sadrži interval slanja i naredbe na čekanju.
POST	`/api/commands/<id>/result`	Slanje rezultata izvršene naredbe natrag poslužitelju.
GET	`/api/hosts`	Popis svih nadziranih računala s trenutnim statusom.
GET	`/api/hosts/<id>/metrics`	Povijesni podatci pokazatelja za odabrano računalo i vremenski raspon.
GET	`/api/alerts`	Popis generiranih upozorenja.

Oznaka `<id>` u putanjama krajnjih točaka zamjenjuje se odgovarajućim identifikatorom: u putanji `/api/commands/<id>/result` riječ je o identifikatoru naredbe, a u putanji `/api/hosts/<id>/metrics` o identifikatoru nadziranog računala.

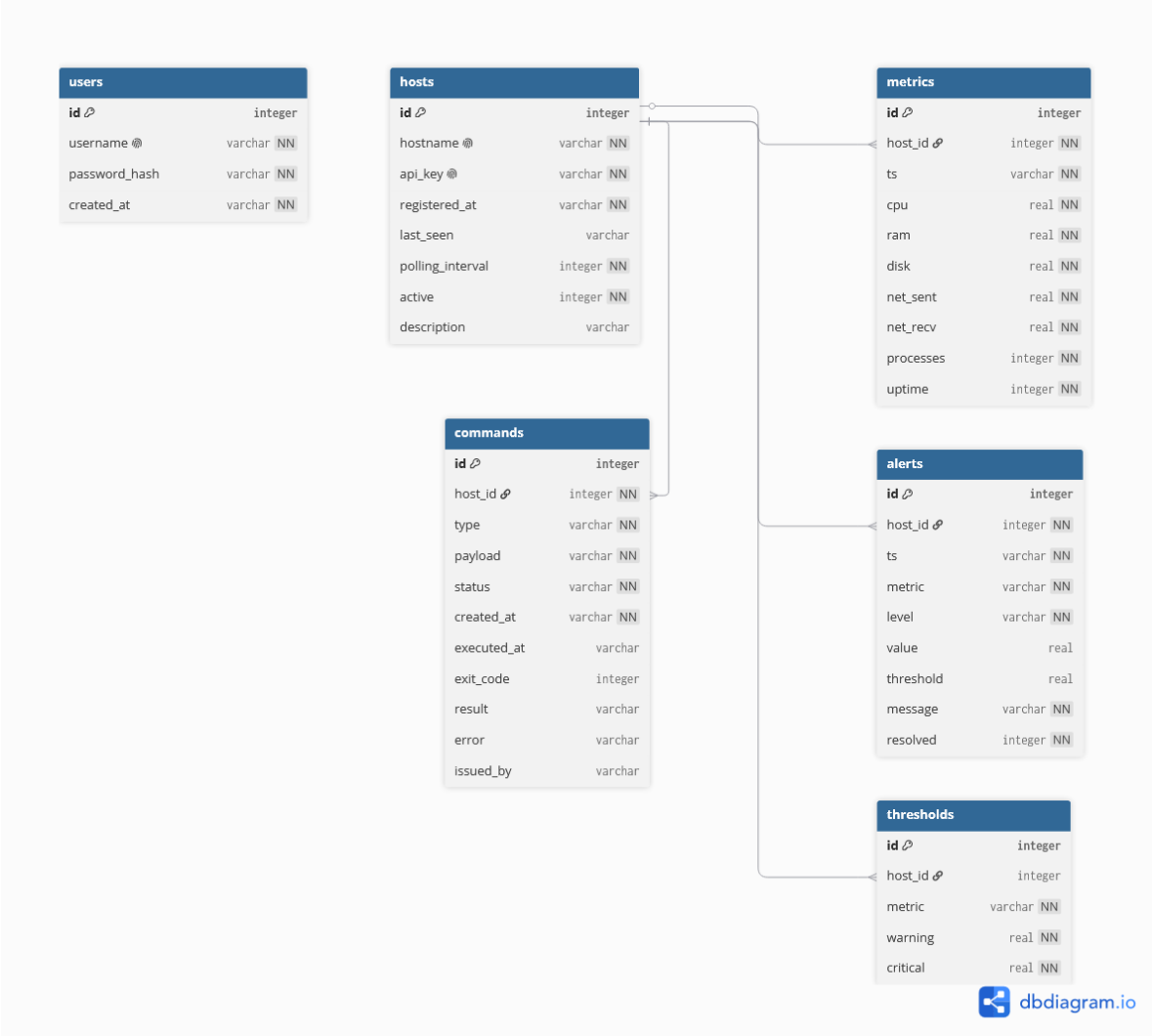
3.3. Model baze podataka

Model podataka sastoji se od šest tablica čiji je pregled dan u tablici 2. Središnja je tablica `hosts`, na koju se stranim ključevima vežu tablice `metrics`, `alerts`, `thresholds` i `commands`, uz kaskadno brisanje pri uklanjanju računala iz nadzora. Zbog velikoga broja zapisa pokazatelja nad tablicom `metrics` izgrađen je složeni indeks po stupcima `host_id` i `ts` koji ubrzava dohvat povijesnih podataka za pojedino računalo. Pragovi upozorenja mogu biti globalni, odnosno vrijediti za sva računala, ili definirani po pojedinom računalu, pri čemu zapis po računalu ima prednost pred globalnim.

Tablica 2: Tablice baze podataka

Tablica	Namjena
`users`	Administratorski korisnički računi web sučelja.
`hosts`	Popis nadziranih računala s ključem API, intervalom slanja i statusom.
`metrics`	Vremenski zapisi sistemskih pokazatelja koje šalju agenti.
`thresholds`	Pragovi upozorenja (`warning` i `critical`) po pokazatelju, globalni ili po računalu.
`alerts`	Generirana upozorenja s razinom, vrijednošću i statusom razrješenja.
`commands`	Red udaljenih naredbi koje web sučelje upućuje agentima.

Odnosi među navedenim tablicama prikazani su dijagramom E-R na slici 2.

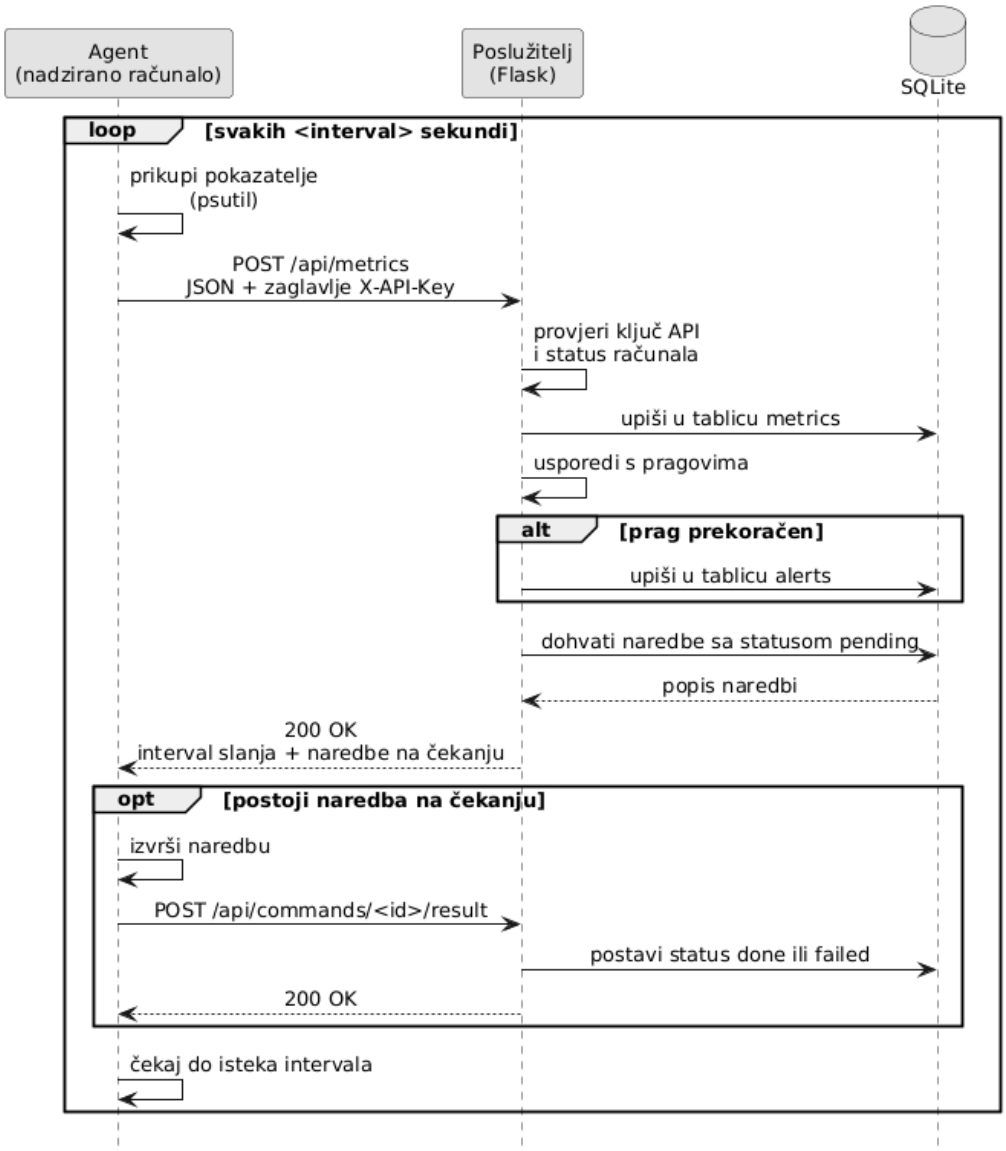


Slika 2: Dijagram E-R baze podataka

3.4. Tijek podataka kroz sustav

Rad sustava temelji se na ponavljajućem ciklusu razmjene poruka između agenta i poslužitelja. Agent najprije prikuplja pokazatelje, zatim ih u formatu JSON šalje na krajnju točku `/api/metrics` zajedno s ključem API u zaglavlju `X-API-Key`. Poslužitelj provjerava valjanost ključa i status računala, pohranjuje zaprimljene vrijednosti u tablicu `metrics` te ih uspoređuje s važećim pragovima upozorenja.

U odgovoru poslužitelj agentu vraća trenutačno važeći interval slanja i popis naredbi koje čekaju na izvršavanje. Ako naredbe postoje, agent ih izvršava i rezultat vraća na krajnju točku `/api/commands/<id>/result`. Nakon toga agent čeka do isteka intervala i ciklus se ponavlja. Opisani slijed razmjene poruka prikazan je na slici 3.



Slika 3: Slijed razmjene poruka između agenta i poslužitelja

3.5. Nefunkcionalni zahtjevi i ograničenja

Uz funkcionalnosti navedene u zadatku rada, na oblikovanje sustava presudno su utjecali i zahtjevi koji se ne odnose izravno na ono što sustav radi, nego na to kako radi. Prvi je zahtjev jednostavnost postavljanja: sustav se mora moći pokrenuti bez posebne poslužiteljske infrastrukture, pa su svi podatci smješteni u jednu datoteku baze podataka, a agent je izveden kao jedna skripta koja ovisi samo o dvjema bibliotekama izvan standardne biblioteke jezika Python.

Drugi je zahtjev otpornost na privremeni prekid mreže. Budući da se agent izvodi na računalu nad kojim administrator nema stalni nadzor, njegov rad ne smije prekinuti nedostupnost poslužitelja; zato su sve mrežne iznimke obrađene, a agent nakon isteka intervala jednostavno pokušava ponovno. Treći je zahtjev mali utrošak resursa na nadziranom računalu, zbog čega agent u jednom prolazu obavlja samo očitanje pokazatelja i jedan zahtjev HTTP, bez lokalne pohrane i bez dodatnih dretvi.

Četvrti zahtjev proizlazi iz mrežnih ograničenja tipičnih okruženja: nadzirana računala često se nalaze iza vatrozida ili prevoditelja mrežnih adresa, pa ne mogu primati dolazne veze. Zbog toga svu komunikaciju započinje agent, a poslužitelj mu odgovore i naredbe prosljeđuje u odgovoru na već uspostavljenu vezu.

Uz navedene zahtjeve, pri projektiranju su svjesno prihvaćena i određena ograničenja. Baza podataka SQLite dopušta samo jednoga pisca u danom trenutku, pa sustav nije predviđen za okruženja s velikim brojem agenata koji istodobno šalju pokazatelje; prema procjeni izvedbe, primjeren je za red veličine od nekoliko desetaka nadziranih računala. Komunikacija se odvija protokolom HTTP unutar lokalne mreže, pa sustav u sadašnjem obliku pretpostavlja da je sama mreža povjerljiva. Konačno, sustav prikuplja unaprijed određen skup pokazatelja i ne omogućuje korisniku definiranje vlastitih mjerenja, što ga razlikuje od općenitih nadzornih rješenja spomenutih u uvodu.

3.6. Format poruka između agenta i poslužitelja

Prethodna potpoglavlja opisuju koje krajnje točke sustav nudi i kojim se redoslijedom pozivaju, ali ne i što se u tim pozivima prenosi. Budući da je razmjena poruka jedina dodirna točka dviju komponenti, njezin je oblik ujedno i ugovor kojega se obje strane moraju držati, pa je u nastavku opisan svaki od triju tipova poruka.

Poruka registracije najjednostavnija je i jedina koja se šalje bez ključa API. Agent u njoj prenosi samo naziv računala i neobavezan opis. Poslužitelj odgovara statusom 201 i porukom koja sadrži dodijeljeni identifikator računala, potvrđeni naziv, novogenerirani ključ API i početni interval slanja. Agent iz odgovora preuzima ključ, identifikator i interval te ih zapisuje u konfiguracijsku datoteku, čime registracija postaje jednokratan postupak: pri svakom sljedećem pokretanju agent već raspolaže ključem i preskače ovaj korak.

Poruka s pokazateljima najčešća je poruka u sustavu i sadrži sedam polja, po jedno za svaki prikupljeni pokazatelj: `cpu`, `ram`, `disk`, `net_sent`, `net_recv`, `processes` i `uptime`. Poruka namjerno ne sadrži ni vremensku oznaku ni identifikator računala. Vremensku oznaku pridjeljuje poslužitelj u trenutku zaprimanja, čime se izbjegava ovisnost o točnosti sata na nadziranom računalu i mogućnost da zapisi različitih računala budu međusobno vremenski nesuglasni. Identifikator računala poslužitelj izvodi iz ključa API u zaglavlju `X-API-Key`, pa se agent ne može lažno predstaviti navođenjem tuđega identifikatora.

Odgovor na tu poruku nosi više informacija nego sam zahtjev. Uz potvrdu o uspjehu sadrži trenutačno važeći interval slanja, oznaku je li računalo i dalje aktivno te popis naredbi koje čekaju na izvršavanje. Time jedan jedini zahtjev agenta obavlja tri posla odjednom: predaje mjerenja, preuzima izmijenjenu konfiguraciju i preuzima naredbe. Upravo je ta objedinjenost razlog zbog kojega nadzirana računala ne moraju biti dostupna izvana, kako je opisano u potpoglavlju 3.1. Pregled arhitekture.

Poruka s rezultatom naredbe sadrži konačni status izvršavanja, izlazni kod, sam rezultat i, ako je došlo do pogreške, njezin opis. Oblik rezultata ovisi o vrsti naredbe: za dohvat popisa procesa riječ je o polju zapisa o procesima, a za izvršavanje proizvoljne naredbe o njezinu standardnom izlazu. Agent tu poruku šalje na krajnju točku u čijoj se putanji nalazi identifikator naredbe, pa poslužitelj prije pohrane može provjeriti pripada li naredba doista računalu koje šalje rezultat.

Sve tri poruke koriste format JSON i skup polja koji je unaprijed određen. Takav izbor donosi dvije praktične prednosti: poruke su čitljive pri otkrivanju pogrešaka bez posebnih alata, a rad sučelja moguće je ispitati običnim klijentom HTTP, bez pokretanja agenta. Nedostatak je što svaka izmjena skupa pokazatelja zahtijeva usklađenu izmjenu obiju komponenti; kako obje nastaju u istom projektu, to u ovom radu nije predstavljalo ograničenje.

4. Implementacija sustava

4.1. Agentska aplikacija

Agentska aplikacija izvedena je kao jedna skripta napisana u programskom jeziku Python koja se konfigurira datotekom `agent_config.json`. Pri prvom se pokretanju agent registrira na poslužitelju slanjem naziva računala na krajnju točku `/api/register`, od poslužitelja dobiva jedinstveni ključ API i identifikator te ih trajno pohranjuje u konfiguracijsku datoteku. Svaki se sljedeći zahtjev autenticira tim ključem; ako poslužitelj ključ odbije, agent se automatski pokušava ponovno registrirati. Sadržaj konfiguracijske datoteke prikazan je u ispisu 1.

```
{
  "server_url": "http://192.168.1.76:5000",
  "interval": 3,
  "hostname": "tin-VirtualBox",
  "description": "",
  "api_key": "<api_key>",
  "host_id": 1,
  "allow_remote_commands": true,
  "max_processes_in_list": 200
}
```

Ispis 1: Konfiguracijska datoteka `agent_config.json`

Prikupljanje pokazatelja ostvareno je razredom `MetricsCollector` koji čuva stanje potrebno za izračun brzine mrežnog prometa: pri svakom se očitanju uspoređuju brojači poslanih i primljenih okteta s vrijednostima iz prethodnog očitanja te se razlika dijeli s proteklim vremenom. Ostali pokazatelji, odnosno opterećenje procesora, zauzeće memorije i korijenske particije diska, broj procesa i vrijeme rada sustava, očitavaju se izravno pozivima biblioteke `psutil`, kako je prikazano u ispisu 2.

```
def collect(self) -> dict:
    now = time.time()

    cpu_pct = psutil.cpu_percent(interval=None)
    vm      = psutil.virtual_memory()
    disk    = psutil.disk_usage("/")
```

```
    proc_n  = len(psutil.pids())
    uptime  = int(now - psutil.boot_time())

    net      = psutil.net_io_counters()
    elapsed  = max(now - self._prev_time, 0.001)
    sent_kbs = (net.bytes_sent - self._prev_net.bytes_sent) / 1024.0
/ elapsed
    recv_kbs = (net.bytes_recv - self._prev_net.bytes_recv) / 1024.0
/ elapsed
    self._prev_net  = net
    self._prev_time = now

    return {
        "cpu":       round(cpu_pct, 2),
        "ram":       round(vm.percent, 2),
        "disk":      round(disk.percent, 2),
        "net_sent":  round(max(sent_kbs, 0), 2),
        "net_recv":  round(max(recv_kbs, 0), 2),
        "processes": proc_n,
        "uptime":    uptime,
    }
```

Ispis 2: Metoda za prikupljanje sistemskih pokazatelja

Glavna petlja agenta u svakom prolazu prikupi pokazatelje, pošalje ih poslužitelju i obradi odgovor. Ako odgovor sadrži promijenjeni interval slanja, agent ga primjenjuje i pohranjuje bez potrebe za ponovnim pokretanjem. Trajanje samoga prolaza oduzima se od intervala čekanja kako bi razmak između uzastopnih očitanja ostao ujednačen, a sve se mrežne i ostale iznimke obrađuju tako da privremeni ispad veze ne prekida rad agenta. Glavna petlja prikazana je u ispisu 3.

```
while True:
    loop_start = time.time()
    try:
        data = collector.collect()
        r = requests.post(metrics_url, json=data,
                          headers=headers, timeout=10)
        if r.status_code == 401:
            print("[agent] API key rejected, attempting to re-
register...")
            cfg = register(cfg)
            headers = {"X-API-Key": cfg["api_key"]}
        elif r.ok:
```

```
            resp = r.json()
            new_interval = resp.get("polling_interval")
            if new_interval and new_interval != interval:
                interval = max(int(new_interval), 1)
                cfg["interval"] = interval
                save_config(cfg)
                print(f"[agent] Interval changed to {interval}s.")
            print(f"[sent] cpu={data['cpu']}% ...")

            pending = resp.get("commands") or []
            if pending:
                if cfg.get("allow_remote_commands", True):
                    print(f"[agent] {len(pending)} new command(s) to
execute")
                    for c in pending:
                        handle_command(c, cfg,

cfg["server_url"].rstrip("/"), headers)
                else:
                    print(f"[agent] {len(pending)} command(s)
refused")
                    # ... agent vraća poruku o odbijanju
        else:
            print(f"[agent] Server returned {r.status_code}:
{r.text[:200]}")
    except requests.RequestException as e:
        print(f"[agent] Network error: {e}")
    except Exception as e:
        print(f"[agent] Error: {e}")

    elapsed = time.time() - loop_start
    time.sleep(max(interval - elapsed, 0))
```

Ispis 3: Glavna petlja agentske aplikacije

4.2. Poslužiteljska aplikacija

Poslužiteljska aplikacija podijeljena je u četiri modula. Modul `server.py` sadrži isključivo definicije ruta razvojnog okvira Flask i pretvorbu zahtjeva HTTP u pozive poslovne logike. Modul `services.py` sadrži poslovnu logiku podijeljenu u servisne razrede `HostService`, `MetricsService`, `ThresholdService`, `AlertService` i `CommandService`, uključujući provjeru ulaznih podataka i ograničavanje vrijednosti na

dopuštene raspone. Modul `database.py` jedini izravno pristupa bazi podataka i sadrži sve upite SQL. Modul `alerts.py` sadrži pozadinsku dretvu koja periodično pokreće provjeru dostupnosti računala, opisanu u potpoglavlju 4.3. Sustav upozorenja. Takva podjela olakšava ispitivanje i buduće izmjene, primjerice zamjenu baze podataka.

Pristup stranicama web sučelja zaštićen je mehanizmom `login_required` koji neprijavljene korisnike preusmjerava na stranicu za prijavu. Pri zaprimanju pokazatelja dodatno se provjerava je li računalo označeno kao aktivno; pokazatelje onemogućenih računala poslužitelj odbija. Krajnja točka za zaprimanje pokazatelja prikazana je u ispisu 4.

```
@app.route("/api/metrics", methods=["POST"])
def api_metrics():
    """Agent submits metrics (auth via X-API-Key). The response
carries the
    current polling interval and any commands waiting for the
agent."""
    host, err = authenticated_host()
    if err:
        return err
    if not host["active"]:
        return jsonify({"error": "host is disabled"}), 403

    try:
        MetricsService.ingest(host, request.get_json(silent=True) or
{})
    except ValidationError as e:
        return jsonify({"error": str(e)}), 400

    return jsonify({
        "status": "ok",
        "polling_interval": host["polling_interval"],
        "active": bool(host["active"]),
        "commands": CommandService.pending_for_agent(host["id"]),
    })
```

Ispis 4: Krajnja točka za zaprimanje pokazatelja

Definicija tablice `metrics` i pripadajućega indeksa prikazana je u ispisu 5. Nad tablicom je izgrađen složeni indeks `idx_metrics_host_ts`, i to nad parom stupaca `host_id` i `ts` upravo tim redoslijedom. Gotovo svaki upit nad pokazateljima najprije bira jedno računalo, a zatim unutar njega vremenski raspon, pa indeks s identifikatorom računala

na prvome mjestu omogućuje bazi podataka da odmah suzi pretragu na zapise toga računala i da ih potom, već poredane po vremenu, pročita redom. Obrnuti redoslijed stupaca ne bi imao isti učinak jer vremenska oznaka sama po sebi ne razdvaja računala. Bez indeksa baza bi podataka pri svakom osvježavanju grafa morala pregledati cijelu tablicu `metrics`, koja je ujedno i tablica koja u sustavu najbrže raste.

```
CREATE TABLE IF NOT EXISTS metrics (
    id         INTEGER PRIMARY KEY AUTOINCREMENT,
    host_id    INTEGER NOT NULL,
    ts         TEXT NOT NULL,
    cpu        REAL NOT NULL,
    ram        REAL NOT NULL,
    disk       REAL NOT NULL,
    net_sent   REAL NOT NULL DEFAULT 0,    -- KB/s during interval
    net_recv   REAL NOT NULL DEFAULT 0,    -- KB/s during interval
    processes  INTEGER NOT NULL DEFAULT 0,
    uptime     INTEGER NOT NULL DEFAULT 0, -- seconds since boot
    FOREIGN KEY (host_id) REFERENCES hosts(id) ON DELETE CASCADE
);
CREATE INDEX IF NOT EXISTS idx_metrics_host_ts ON metrics(host_id,
ts);
```

Ispis 5: Definicija tablice `metrics` i pripadajućega indeksa

4.2.1. Provjera i normalizacija ulaznih podataka

Poslužitelj ne pretpostavlja da su podatci koje zaprima od agenta ispravni. Prije pohrane svaka poruka prolazi kroz postupak normalizacije izveden u metodi `MetricsService.normalize`, koja od neobrađenoga sadržaja poruke JSON gradi zapis poznatoga oblika.

Postupak se oslanja na tri pomoćne funkcije. Funkcije `_coerce_float` i `_coerce_int` pretvaraju vrijednost pojedinoga polja u broj i, ako pretvorba nije moguća, podižu iznimku `ValidationError` s porukom u kojoj je naveden naziv problematičnog polja. Funkcija `_clamp` zatim ograničava vrijednost na dopušteni raspon. Pokazatelji izraženi u postotcima, odnosno opterećenje procesora te zauzeće memorije i diska, ograničavaju se na raspon od 0 do 100, dok se brojači koji po prirodi ne mogu biti negativni, odnosno brzine mrežnog prometa, broj procesa i vrijeme rada sustava, ograničavaju odozdo na nulu.

Takva obrada ima dvostruku svrhu. S jedne strane štiti prikaz: jedna neispravna vrijednost, primjerice zauzeće memorije od nekoliko tisuća postotaka nastalo pogreškom u očitanju, promijenila bi mjerilo grafa i učinila cijeli vremenski niz nečitljivim. S druge strane sprječava lažna upozorenja, jer bi takva vrijednost odmah prekoračila kritični prag i stvorila zapis u tablici upozorenja.

Ako normalizacija ne uspije, poslužitelj poruku ne pohranjuje, nego agentu vraća statusni kod 400 zajedno s porukom o pogrešci, pa agent iz odgovora može razaznati koje je polje neispravno. Provjera se namjerno izvodi na poslužitelju, a ne u agentu: agent se izvodi na nadziranom računalu i njegova se konfiguracija može izmijeniti, dok je poslužitelj jedina komponenta koja piše u bazu podataka i stoga jedino mjesto na kojem se ispravnost podataka može pouzdano jamčiti.

4.3. Sustav upozorenja

Za pokazatelje opterećenja procesora te zauzeća memorije i diska definiraju se dva praga: prag upozorenja (engl. *warning*) i kritični prag (engl. *critical*). Pri svakom zaprimanju pokazatelja poslužitelj uspoređuje vrijednosti s važećim pragovima te po potrebi generira upozorenje s pripadajućom razinom, izmjerenom vrijednošću i prekoračenim pragom. Kako kontinuirano prekoračenje praga ne bi stvaralo velik broj istovrsnih zapisa, upozorenja se unutar vremenskog prozora od pet minuta ne udvostručuju. Postupak provjere pragova prikazan je u ispisu 6.

```
def evaluate_metric(host_id, hostname, record):
    """Compare a freshly stored metric against the effective
thresholds and
    raise warning or critical alerts where the value crosses a
limit."""
    thresholds = db.get_thresholds(host_id)
    for metric, label in AlertService.METRIC_LABELS.items():
        t = thresholds.get(metric)
        if not t:
            continue
        value = record.get(metric, 0)
        level = AlertService._level_for(value, t)
        if level is None:
            continue
        limit = t[level]
```

```
        db.insert_alert(
            host_id, metric, level,
            f"{label} on {hostname} = {value:.1f}% ({level} >=
{limit}%)",
            value=value, threshold=limit,
        )

@staticmethod
def _level_for(value, threshold):
    if value >= threshold["critical"]:
        return "critical"
    if value >= threshold["warning"]:
        return "warning"
    return None

@staticmethod
```

Ispis 6: Provjera pragova i generiranje upozorenja

Nedostupnost računala otkriva pozadinska dretva izvedena u modulu `alerts.py`. Dretva se pokreće zajedno s poslužiteljem, u pozadinskom je načinu rada, pa ne sprječava njegovo zaustavljanje, i svakih trideset sekundi pokreće provjeru `AlertService.check_offline_hosts`. Sama dretva ne sadrži nikakvu poslovnu logiku, nego samo brine o vremenskom okidanju, čime je pravilo o tome što se smatra nedostupnošću ostalo na jednome mjestu, u servisnom sloju. Izvedba dretve prikazana je u ispisu 7.

```
import threading
import time

from services import AlertService


def offline_monitor_loop(interval_s: int = 30):
    while True:
        try:
            AlertService.check_offline_hosts()
        except Exception as e:
            print(f"[offline_monitor] error: {e}")
        time.sleep(interval_s)
```

```
def start_background_monitor(interval_s: int = 30):
    t = threading.Thread(target=offline_monitor_loop,
                         args=(interval_s,), daemon=True)
    t.start()
    return t
```

Ispis 7: Pozadinska dretva za otkrivanje nedostupnosti

Provjera preskače onemogućena računala i ona koja se još nikada nisu javila, a za ostala uspoređuje vrijeme posljednjega javljanja s prozorom koji iznosi trostruki interval slanja, uz najmanju vrijednost od trideset sekundi. Donja granica sprječava da računala s vrlo kratkim intervalom budu proglašena nedostupnima zbog jednoga izostalog javljanja. Web sučelje pritom koristi nešto kraću granicu od petnaest sekundi, pa se računalo u pregledu prikaže kao nedostupno prije nego što se generira upozorenje. Sva se upozorenja prikazuju u web sučelju, gdje ih korisnik može označiti razriješenima.

4.4. Udaljeno izvršavanje naredbi

Udaljeno upravljanje ostvareno je redom naredbi u tablici `commands`. Kada korisnik u web sučelju zatraži radnju nad nadziranim računalom, poslužitelj u red upisuje naredbu sa statusom `pending`. Naredbe na čekanju agent dobiva u odgovoru na sljedeće slanje pokazatelja, izvršava ih te rezultat vraća na krajnju točku `/api/commands/<id>/result`, nakon čega naredba prelazi u status `done` ili `failed`. Podržane su tri vrste naredbi: dohvat popisa procesa poredanih po opterećenju procesora, slanje signala `SIGTERM` ili `SIGKILL` odabranom procesu te izvršavanje proizvoljne naredbe s vremenskim ograničenjem.

Budući da udaljeno izvršavanje naredbi predstavlja sigurnosno osjetljivu funkcionalnost, agent je može u cijelosti onemogućiti postavkom `allow_remote_commands` u konfiguracijskoj datoteci. U tom slučaju agent zaprimljene naredbe odbija i poslužitelju vraća odgovarajuću poruku o pogrešci. Poslužitelj dodatno provjerava pripada li rezultat naredbe upravo onomu računalu kojemu je naredba upućena, a naredbe koje predugo ostanu u statusu izvršavanja (pet minuta) automatski se označavaju neuspjelima.

4.5. Sigurnosni mehanizmi

Lozinke korisnika web sučelja ne pohranjuju se u izvornom obliku, nego kao sažetak izveden funkcijom PBKDF2-SHA256 s nasumično generiranom soli [7], čime se otežava otkrivanje lozinki u slučaju neovlaštenog pristupa bazi podataka. Sesije prijavljenih korisnika zaštićene su tajnim ključem aplikacije koji se učitava iz varijable okoline, pa se ne nalazi u programskom kôdu ni u repozitoriju. Pri prvom pokretanju poslužitelj stvara unaprijed postavljeni administratorski račun i na standardnom izlazu ispisuje upozorenje da mu je lozinku potrebno promijeniti; u primjeni izvan ispitnog okruženja taj je korak obvezan.

Agenti se autenticiraju jedinstvenim ključevima API generiranima pri registraciji, pa poslužitelj svaki zaprimljeni pokazatelj i rezultat naredbe može jednoznačno povezati s računalom kojemu ključ pripada. Komunikacija se u ispitnom okruženju odvija protokolom HTTP unutar lokalne mreže; za primjenu izvan takva okruženja preporučuje se uvođenje protokola HTTPS, što je navedeno među mogućim poboljšanjima u zaključku rada.

4.6. Dohvat povijesnih podataka

Grafovi na stranici s detaljnim prikazom računala crtaju se iz vremenskoga niza pokazatelja koji web sučelje dohvaća s krajnje točke `/api/hosts/<id>/metrics`. Korisnik uz zahtjev navodi željeni vremenski raspon, a poslužitelj ga obrađuje u dvama koracima.

Metoda `MetricsService.range_to_since` pretvara oznaku raspona iz web sučelja u apsolutni trenutak od kojega se podatci dohvaćaju. Podržane su oznake za jedan sat, šest sati, jedan dan i sedam dana, a ako zahtjev sadrži nepoznatu oznaku, primjenjuje se unaprijed postavljeni raspon od jednoga sata. Time se izbjegava potreba da klijent poslužitelju šalje apsolutna vremena i pripadajuće provjere.

Metoda `MetricsService.history` zatim dohvaća zapise iz baze podataka, uz ograničenje broja vraćenih točaka. Unaprijed je postavljeno da se vraća najviše 500 zapisa, a najveći dopušteni broj iznosi 5000, pri čemu se i sam zahtijevani broj prethodno provjerava istim mehanizmom koji se koristi pri normalizaciji pokazatelja. Ograničenje je uvedeno zato što pri intervalu slanja od tri sekunde raspon od sedam dana obuhvaća više od dvjesto tisuća zapisa: prijenos takve količine podataka usporio bi otvaranje stranice, a graf širine nekoliko

stotina slikovnih točaka ionako ne može smisleno prikazati više podataka nego što ima vodoravnih točaka.

Upit kojim se dohvaćaju povijesni podatci ograničava zapise po nadziranom računalu i po vremenu, pa se u potpunosti oslanja na složeni indeks `idx_metrics_host_ts` opisan u potpoglavlju 4.2. Poslužiteljska aplikacija.

Dohvaćeni niz poslužitelj vraća u formatu JSON, a jezik JavaScript na stranici iz njega gradi skupove podataka koje biblioteka `Chart.js` iscrtava u trima grafovima. Budući da se osvježavanje izvodi periodičnim ponavljanjem istoga zahtjeva, promjena vremenskog raspona u sučelju ne zahtijeva ponovno učitavanje stranice.

4.7. Održavanje baze podataka

Tablica `metrics` jedina je tablica u sustavu koja neprekidno raste. Pri unaprijed postavljenom intervalu slanja od tri sekunde jedno nadzirano računalo dnevno stvara oko 28 800 zapisa, pa deset računala u mjesec dana proizvede blizu devet milijuna zapisa. Bez povremenoga čišćenja datoteka baze podataka postala bi nepotrebno velika, a upiti nad njom sporiji.

Zbog toga je u modulu `database.py` predviđena funkcija `prune_old_metrics`, koja briše sve zapise starije od zadanoga broja dana, koji je unaprijed postavljen na trideset. U trenutačnoj izvedbi funkcija je pripremljena, ali se ne poziva automatski, nego ju je potrebno pokrenuti ručno ili je uključiti u pozadinsku dretvu poslužitelja; njezino periodično pozivanje navedeno je među mogućim poboljšanjima sustava.

Drugi je oblik održavanja čišćenje reda naredbi. Naredba koju je agent preuzeo prelazi u status izvršavanja i u njemu ostaje sve dok agent ne pošalje rezultat. Ako se agent u međuvremenu zaustavi ili izgubi mrežnu vezu, naredba bi u tom stanju ostala trajno. Funkcija `cleanup_stuck_commands` zato sve naredbe koje su u statusu izvršavanja dulje od pet minuta označava neuspjelima, a poziva se pri svakom otvaranju stranice s procesima, dakle upravo ondje gdje bi zaostala naredba korisniku bila vidljiva.

Osim složenog indeksa nad tablicom `metrics` opisanog u potpoglavlju 4.2. Poslužiteljska aplikacija, radi bržega su dohvata izgrađeni i indeks `idx_alerts_host_ts`

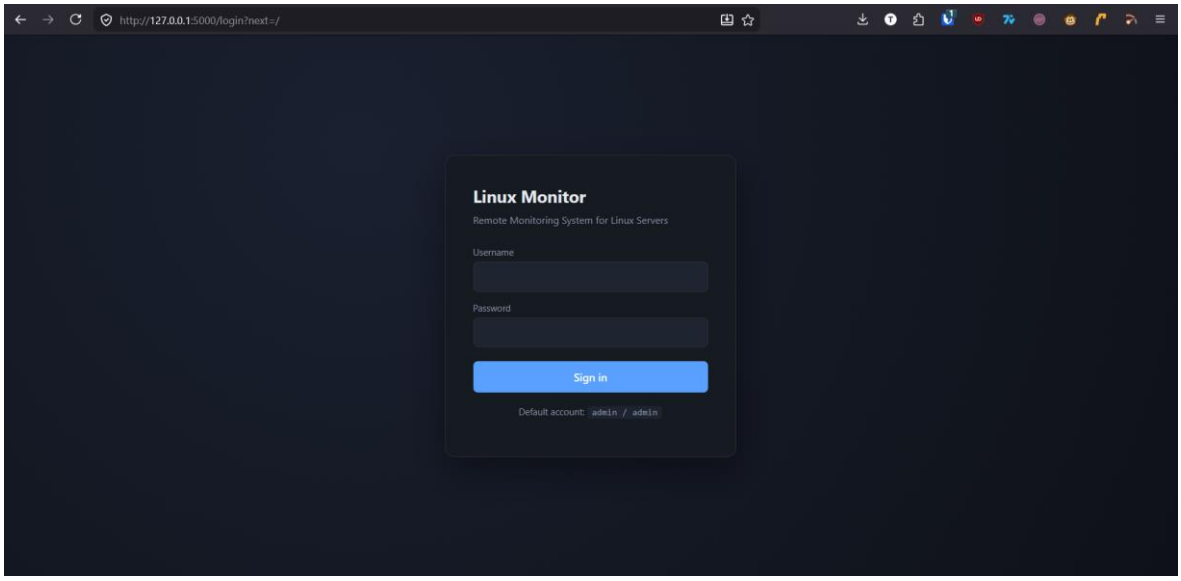
nad tablicom `alerts`, koji ubrzava prikaz upozorenja pojedinoga računala poredanih po vremenu, te indeksi `idx_cmd_host_status` i `idx_cmd_host_created` nad tablicom `commands`. Prvi ubrzava dohvat naredbi na čekanju, koji se izvodi pri svakom javljanju agenta i time je najčešći upit u sustavu, a drugi prikaz povijesti naredbi pojedinoga računala u web sučelju.

5. Korisničko sučelje

Korisničko sučelje izvedeno je kao skup stranica generiranih predlošcima Jinja2 sa zajedničkim osnovnim predloškom koji sadrži navigaciju. Prikazi stanja osvježavaju se periodičnim pozivima programskog sučelja iz jezika JavaScript, bez ponovnog učitavanja stranice.

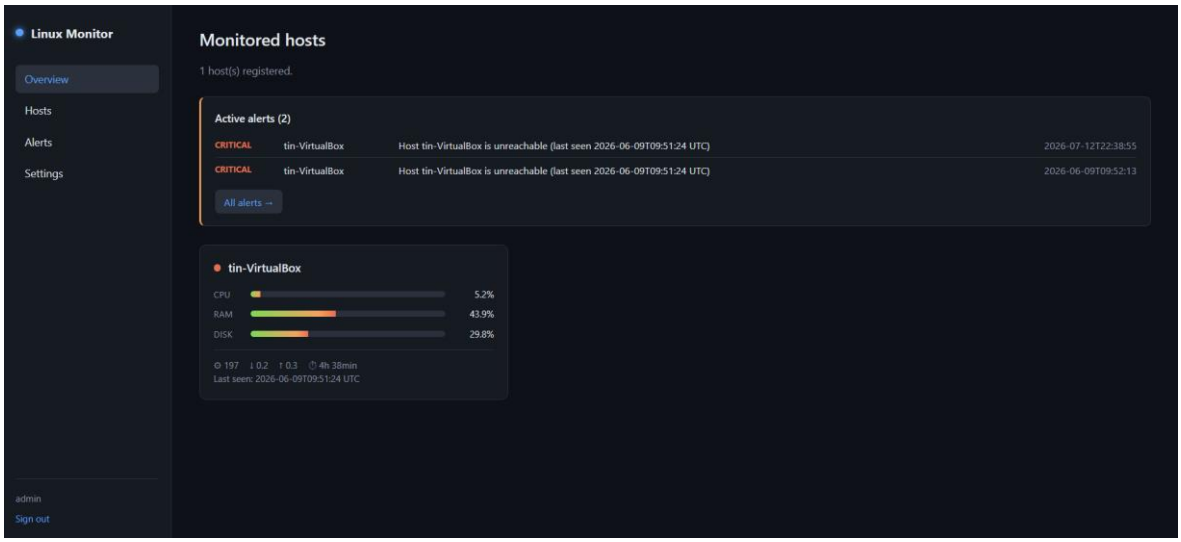
5.1. Prijava korisnika i nadzorna ploča

Pristup aplikaciji započinje stranicom za prijavu prikazanom na slici 4. Neprijavljeni se korisnik pri pokušaju otvaranja bilo koje druge stranice preusmjerava na tu stranicu.



Slika 4: Stranica za prijavu korisnika

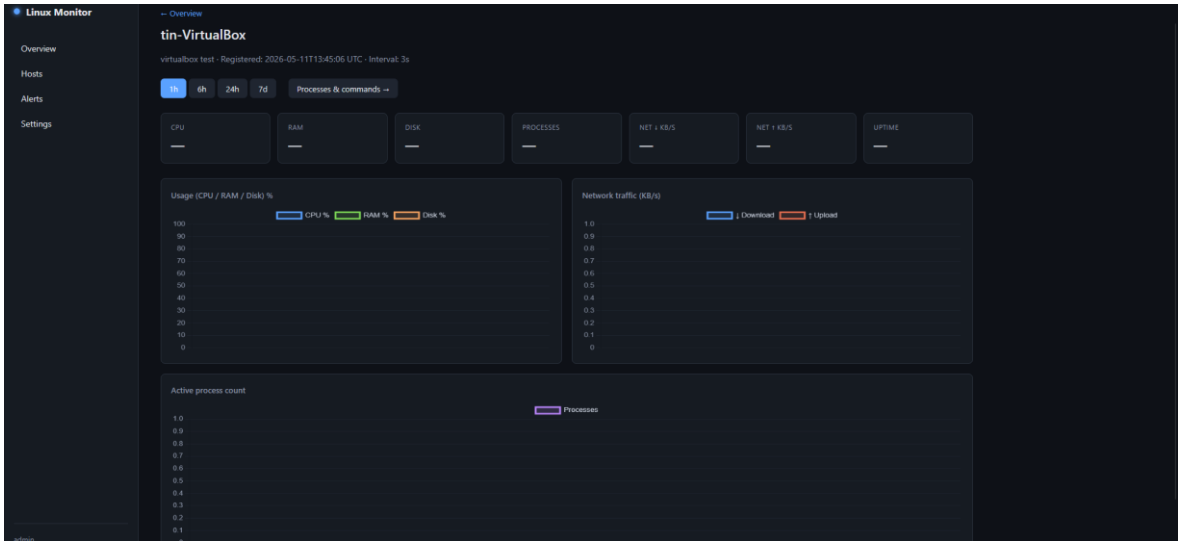
Nakon uspješne prijave korisniku se prikazuje nadzorna ploča s pregledom svih nadziranih računala, njihovim trenutačnim statusom (dostupno ili nedostupno), posljednjim izmjerenim vrijednostima pokazatelja i brojem nerazriješenih upozorenja, kao na slici 5.



Slika 5: Nadzorna ploča s pregledom nadziranih računala

5.2. Detaljni prikaz nadziranog računala

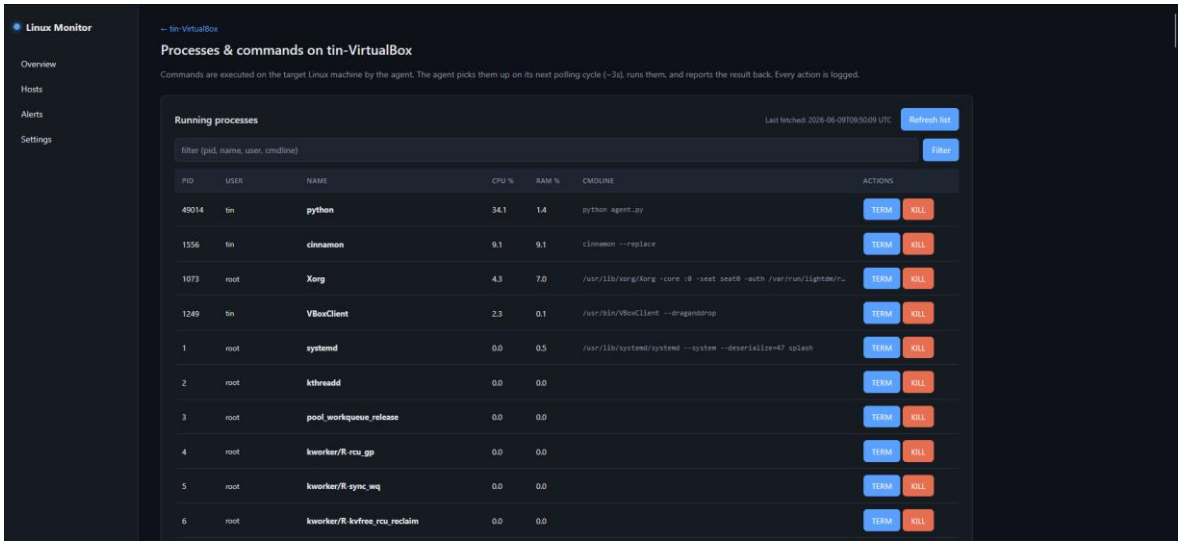
Odabirom pojedinoga računala otvara se stranica s detaljnim prikazom. Na njoj se nalaze tri grafa izrađena bibliotekom `Chart.js`: graf pokazatelja opterećenja procesora te zauzeća memorije i diska, graf mrežnog prometa i graf broja procesa. Korisnik može odabrati vremenski raspon prikaza povijesnih podataka, a grafovi se automatski osvježavaju novim mjerenjima, kao što je prikazano na slici 6.



Slika 6: Detaljni prikaz računala s grafovima pokazatelja

5.3. Upravljanje procesima

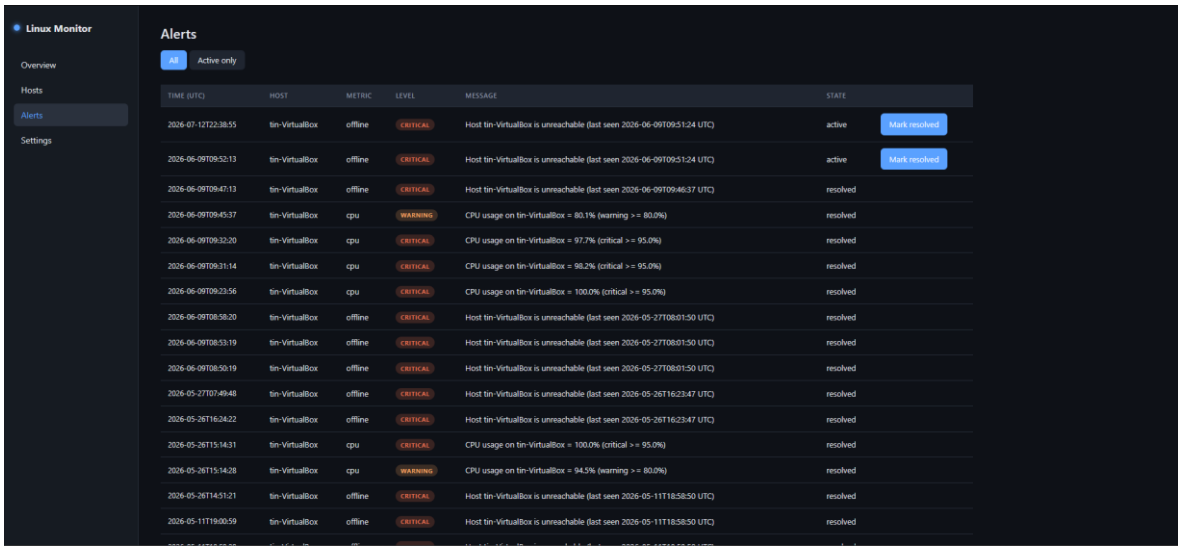
Stranica s procesima prikazuje popis procesa nadziranog računala dohvaćen udaljenom naredbom, uz mogućnost pretraživanja te slanja signala za završetak odabranog procesa. Popis sadrži identifikator procesa, naziv, korisnika te udio zauzetog procesorskog vremena i memorije, kao što je prikazano na slici 7.



Slika 7: Popis procesa nadziranog računala

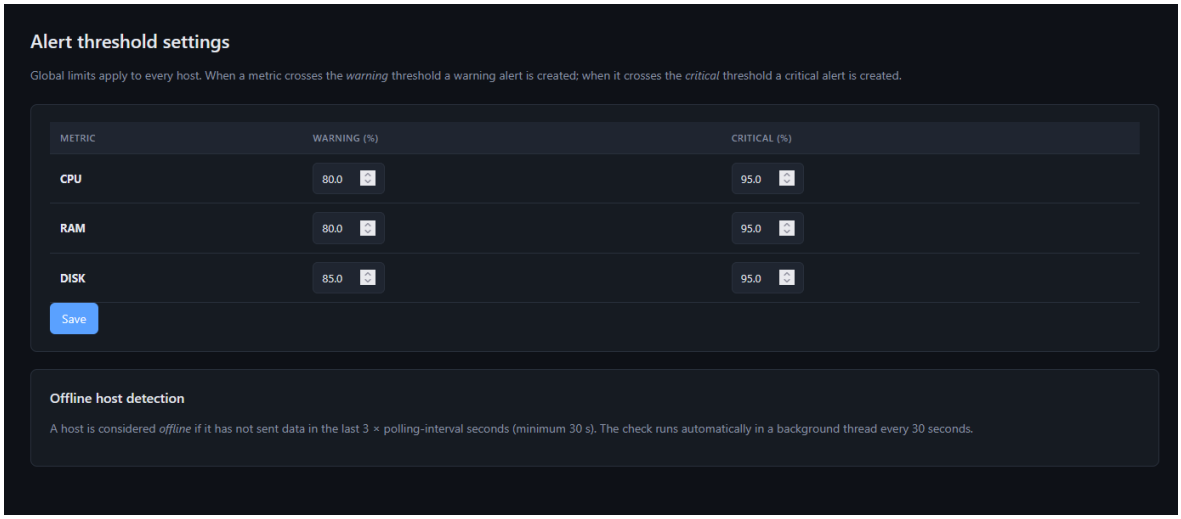
5.4. Upozorenja i postavke pragova

Stranica upozorenja daje kronološki popis svih generiranih upozorenja s razinom i vrijednošću koja je prag prekoračila te mogućnošću razrješavanja pojedinog upozorenja. Prikaz je vidljiv na slici 8.



Slika 8: Pregled generiranih upozorenja

Na stranici postavki uređuju se globalni pragovi upozorenja i pragovi po pojedinom računalu, pri čemu vrijednost definirana za pojedino računalo ima prednost pred globalnom, kao što prikazuje slika 9.



Slika 9: Stranica za uređivanje pragova upozorenja

5.5. Upravljanje nadziranim računalima

Posljednja stranica web sučelja namijenjena je održavanju popisa nadziranih računala. U tablici se za svako računalo prikazuju oznaka dostupnosti, naziv, opis, interval slanja, oznaka aktivnosti, trenutak posljednjega javljanja i skraćeni prikaz ključa API, a uz svaki

redak stoje dugmeta za spremanje izmjena i za brisanje računala. Podatke stranici predaje metoda `HostService.list_with_status`, koja uz pohranjene vrijednosti dodaje i izračunatu oznaku dostupnosti.

Interval slanja može se izmijeniti za svako računalo zasebno. Unesena se vrijednost ograničava na raspon od jedne sekunde do jednoga sata, i to na dvama mjestima: u samome obrascu te ponovno u servisnom sloju pri obradi zahtjeva. Nova vrijednost ne šalje se agentu odmah, nego ju agent preuzima u odgovoru na prvo sljedeće javljanje i trajno je zapisuje u svoju konfiguracijsku datoteku.

Računalo se može i privremeno onemogućiti. Takvo računalo ostaje u bazi podataka zajedno sa svim prikupljenim podatcima, ali poslužitelj njegove pokazatelje odbija statusom 403, a provjera dostupnosti, opisana u potpoglavlju 4.3. Sustav upozorenja, preskače ga, pa privremeno isključeno računalo ne stvara upozorenja o nedostupnosti. Ta je mogućnost korisna pri planiranom održavanju nadziranog računala.

Trajno uklanjanje računala briše samo zapis iz tablice `hosts`, a zbog stranih ključeva s kaskadnim brisanjem, opisanih u potpoglavlju 3.3. Model baze podataka, istodobno se uklanjaju i svi pripadajući pokazatelji, upozorenja, pragovi i naredbe. Valja napomenuti da SQLite provodi kaskadno brisanje samo ako je na vezi izričito uključeno, pa se pri otvaranju svake veze prema bazi izvršava naredba `PRAGMA foreign_keys = ON`. Bez nje bi definicije stranih ključeva bile prihvaćene, ali se ne bi primjenjivale, a u bazi bi nakon brisanja računala ostali zapisi bez matičnoga računala.

Ista stranica omogućuje i ručno dodavanje računala. Poslužitelj u tom slučaju odmah generira ključ API i prikazuje ga korisniku, koji ga zatim upisuje u konfiguracijsku datoteku agenta. Taj je postupak obrnut od uobičajenoga, u kojem ključ traži sam agent, a koristan je kada se agent postavlja na računalo koje u trenutku pripreme još nema pristup poslužitelju. Ako se pritom navede naziv računala koje već postoji, poslužitelj mu dodjeljuje novi ključ i ponovno ga označava aktivnim, pa ista radnja ujedno služi i za zamjenu ključa računalu kojemu je stari ključ kompromitiran ili izgubljen.

6. Ispitivanje sustava

Sustav je ispitan u kontroliranom okruženju sastavljenom od poslužitelja i virtualnog stroja s operativnim sustavom Linux na kojemu je pokrenut agent. Ispitivanje je obuhvatilo provjeru točnosti prikupljenih podataka, rad sustava upozorenja, stabilnost komunikacije te ispravnost udaljenog izvršavanja naredbi.

6.1. Ispitno okruženje

Ispitno je okruženje postavljeno na jednom fizičkom računalu pomoću alata za virtualizaciju. Sastav okruženja, s ulogama pojedinih virtualnih strojeva i dodijeljenim resursima, prikazan je u tablici 3.

Tablica 3: Sastav ispitnog okruženja

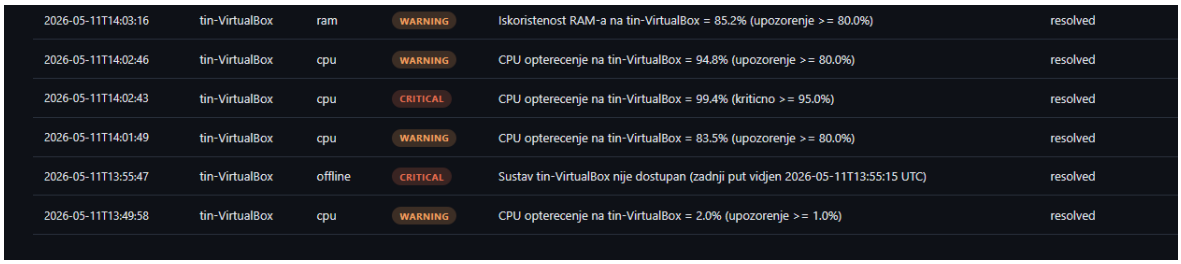
Uloga	Operativni sustav	Dodijeljeni resursi	Interval slanja
Središnji poslužitelj	Windows	—	—
Nadzirano računalo	Linux Mint	2048 MB memorije, jedna procesorska jezgra	3 s

6.2. Provjera točnosti prikupljenih podataka

Točnost prikupljenih pokazatelja provjerena je usporedbom vrijednosti prikazanih u web sučelju s vrijednostima koje na nadziranom računalu prikazuju standardni sistemski alati `top`, `free` i `df`. Vrijednosti su očitavane istodobno, u više ponavljanja i pri različitim razinama opterećenja računala. Odstupanja su bila u granicama očekivanima zbog različitih trenutaka uzorkovanja, čime je potvrđeno da agent ispravno očitava stanje sustava.

6.3. Ispitivanje sustava upozorenja

Rad sustava upozorenja provjeren je namjernim izazivanjem prekoračenja pragova. Opterećenje procesora povećano je pokretanjem računski zahtjevnoga procesa, a zauzeće memorije zauzimanjem većega memorijskog bloka. U oba je slučaja poslužitelj generirao upozorenje odgovarajuće razine, koje se potom pojavilo na nadzornoj ploči i na stranici upozorenja, kao što prikazuje slika 10.



2026-05-11T14:03:16	tin-VirtualBox	ram	WARNING	Iskoristenost RAM-a na tin-VirtualBox = 85.2% (upozorenje >= 80.0%)	resolved
2026-05-11T14:02:46	tin-VirtualBox	cpu	WARNING	CPU opterecenje na tin-VirtualBox = 94.8% (upozorenje >= 80.0%)	resolved
2026-05-11T14:02:43	tin-VirtualBox	cpu	CRITICAL	CPU opterecenje na tin-VirtualBox = 99.4% (kriticno >= 95.0%)	resolved
2026-05-11T14:01:49	tin-VirtualBox	cpu	WARNING	CPU opterecenje na tin-VirtualBox = 83.5% (upozorenje >= 80.0%)	resolved
2026-05-11T13:55:47	tin-VirtualBox	offline	CRITICAL	Sustav tin-VirtualBox nije dostupan (zadnji put vidjen 2026-05-11T13:55:15 UTC)	resolved
2026-05-11T13:49:58	tin-VirtualBox	cpu	WARNING	CPU opterecenje na tin-VirtualBox = 2.0% (upozorenje >= 1.0%)	resolved

Slika 10: Upozorenja generirana tijekom ispitivanja opterećenja

Provjereno je i sprječavanje udvostručavanja upozorenja: pri trajnom prekoračenju praga tijekom nekoliko uzastopnih očitanja generirano je samo jedno upozorenje unutar vremenskog prozora od pet minuta. Otkrivanje nedostupnosti računala ispitano je isključivanjem virtualnoga stroja, nakon čega je poslužitelj u očekivanom roku generirao upozorenje o nedostupnosti.

6.4. Ispitivanje stabilnosti komunikacije

Stabilnost komunikacije ispitana je privremenim prekidanjem mrežne veze između agenta i poslužitelja. Tijekom prekida agent je nastavio s radom, a pokušaji slanja završavali su obrađenom iznimkom bez prekida izvođenja. Nakon ponovne uspostave veze agent je nastavio slati pokazatelje bez intervencije korisnika. Ispitana je i izmjena intervala slanja iz web sučelja, pri čemu je agent novi interval primijenio već pri sljedećem javljanju.

6.5. Ispitivanje udaljenog izvršavanja naredbi

Udaljeno izvršavanje naredbi ispitano je dohvatom popisa procesa nadziranog računala te slanjem signala `SIGTERM` odabranom procesu, nakon čega je proces nestao s osvježenoga popisa. Provjereno je i ponašanje sustava kada je u konfiguraciji agenta postavka `allow_remote_commands` isključena: agent je naredbu odbio, a poslužitelj ju je označio statusom `failed` uz pripadajuću poruku o pogrešci.

6.6. Pregled rezultata ispitivanja

Objedinjeni pregled provedenih ispitivanja, načina provedbe i dobivenih rezultata dan je u tablici 4.

Tablica 4: Pregled provedenih ispitivanja i rezultata

Ispitivanje	Način provedbe	Rezultat
Prekoračenje pragova	Umjetno opterećenje procesora i memorije	Upozorenja generirana s ispravnom razinom
Udvostručavanje upozorenja	Trajno prekoračenje praga	Jedno upozorenje unutar prozora od pet minuta
Nedostupnost računala	Isključivanje virtualnog stroja	Upozorenje o nedostupnosti generirano u očekivanom roku
Stabilnost komunikacije	Privremeni prekid mrežne veze	Agent nastavlja rad bez intervencije
Izmjena intervala slanja	Promjena vrijednosti u web sučelju	Novi interval primijenjen pri sljedećem javljanju
Udaljene naredbe	Dohvat procesa i slanje signala `SIGTERM`	Naredbe izvršene, status `done`
Onemogućene naredbe	Postavka `allow_remote_commands` isključena	Naredba odbijena, status `failed`

Rezultati prikazani u tablici 4 pokazuju da su sve predviđene funkcionalnosti sustava ispravno izvedene te da sustav stabilno radi i u uvjetima privremenoga gubitka mrežne veze.

6.7. Automatizirano ispitivanje

Uz ručna ispitivanja opisana u prethodnim potpoglavljima, uz sustav je izrađen i skup automatiziranih provjera u datoteci `tests.py`. Provjere se izvode nad poslužiteljskom aplikacijom bez pokretanja pravoga agenta i bez pokretanja mrežnoga poslužitelja: koristi se ispitni klijent razvojnog okvira Flask, koji zahtjeve HTTP prosljeđuje izravno aplikaciji. Prije učitavanja ostalih modula putanja baze podataka preusmjerava se u privremeni direktorij, čime je zajamčeno da ispitivanje ni u kojem slučaju ne dira stvarne podatke.

Takav pristup omogućuje da se u nekoliko sekundi provjere upravo oni rubni slučajevi koje je ručnim ispitivanjem teško izazvati, primjerice pokazatelj izvan dopuštenoga raspona ili neispravan ključ API. Izvadak iz skupa provjera prikazan je u ispisu 8.

```
r = client.post("/api/register",
                json={"hostname": "vm-test", "description": "ci"})
check(r.status_code == 201, "agent registration returns 201")
key = r.get_json()["api_key"]
host_id = r.get_json()["host_id"]

r = client.post("/api/metrics", headers={"X-API-Key": key},
```

```
                json={"cpu": 150, "ram": -5, "disk": 99,
                      "net_sent": -10, "processes": 200, "uptime":
3600})
check(r.status_code == 200, "metric submission accepted")

latest = MetricsService.history(host_id, "1h")[-1]
check(latest["cpu"] == 100.0, "cpu of 150 is clamped to 100")
check(latest["ram"] == 0.0, "ram of -5 is clamped to 0")
check(latest["net_sent"] == 0.0, "negative net_sent is floored to
0")

check(any(a["metric"] == "cpu" and a["level"] == "critical"
```

Ispis 8: Izvadak iz skupa automatiziranih provjera

Skup obuhvaća četrnaest provjera. Uz registraciju agenta i zaprimanje pokazatelja provjerava se normalizacija ulaznih podataka opisana u potpoglavlju 4.2.1. Provjera i normalizacija ulaznih podataka, i to slanjem opterećenja procesora od 150 posto, negativnoga zauzeća memorije i negativne brzine mrežnog prometa; sve tri vrijednosti moraju biti svedene na granice dopuštenoga raspona. Provjerava se i da prekoračenje kritičnoga praga stvara upozorenje odgovarajuće razine te da poslužitelj neispravan ključ API odbija statusom 401.

Ostatak provjera pokriva ograničenja poslovne logike: prag upozorenja ne smije biti veći ili jednak kritičnomu pragu, ne smije se poslati signal procesu s identifikatorom manjim ili jednakim jedan, prazna se naredba odbija, predugo vrijeme izvršavanja svodi se na najveću dopuštenu vrijednost, ponovljeni zahtjev za osvježavanje popisa procesa ne stvara drugu naredbu u redu, naredbe na čekanju predaju se agentu u odgovoru na sljedeće javljanje, a računalo koje se upravo javilo prikazuje se kao dostupno. Pri izvođenju su sve provjere uspješno prošle.

Vrijednost je toga skupa dvojaka. Prvo, ispitivanje se može ponoviti u svakom trenutku i bez ispitnoga okruženja s virtualnim strojevima, pa se pri svakoj izmjeni kôda brzo uočava je li narušeno neko od postojećih pravila. Drugo, provjere ujedno dokumentiraju očekivano ponašanje sustava na mjestima na kojima je ono definirano ograničenjima, a ne opisom.

6.8. Ograničenja provedenog ispitivanja

Rezultati opisani u prethodnim potpoglavljima potvrđuju da sustav radi kako je predviđeno, ali vrijede unutar granica okruženja u kojem su dobiveni. Te je granice potrebno navesti kako se zaključci ne bi proširili izvan onoga što je doista ispitano.

Najveće ograničenje jest broj nadziranih računala. Ispitivanje je provedeno s jednim agentom, pa ponašanje sustava pri istodobnom javljanju većega broja agenata nije provjereno. To je ograničenje osobito važno zbog izbora baze podataka: SQLite dopušta samo jednoga pisca u danom trenutku, pa bi se pri dovoljno velikom broju agenata s kratkim intervalom slanja mogla pojaviti čekanja pri upisu. Procjena iz potpoglavlja 3.5. Nefunkcionalni zahtjevi i ograničenja, prema kojoj je sustav primjeren za nekoliko desetaka računala, proizlazi iz svojstava korištenih tehnologija, a ne iz mjerenja.

Drugo je ograničenje trajanje ispitivanja. Sustav je promatran u razdobljima od najviše nekoliko sati, dok je tablica pokazatelja sadržavala razmjerno malo zapisa. Utjecaj rasta te tablice na brzinu dohvata povijesnih podataka, kao i na veličinu datoteke baze podataka, stoga nije izmjeren; upravo je zbog toga u potpoglavlju 4.7. Održavanje baze podataka opisano čišćenje starijih zapisa predviđeno kao potreban, ali još neuključen mehanizam.

Treće, ispitivanje je provedeno u jednoj lokalnoj mreži s malim kašnjenjem i bez gubitka paketa. Prekid veze simuliran je potpunim isključenjem mrežnoga sučelja, što je krajnji slučaj; ponašanje pri nepouzdanoj vezi s povremenim gubitkom paketa ili velikim kašnjenjem nije ispitano. Nije provjeren ni rad na drugim distribucijama operativnog sustava Linux osim one navedene u potpoglavlju 6.1. Ispitno okruženje.

Navedena ograničenja ne dovode u pitanje dobivene rezultate, nego određuju smjer daljnjega ispitivanja: mjerenje odziva poslužitelja pri većem broju agenata, dugotrajno praćenje rasta baze podataka te ispitivanje u mreži s namjerno unesenim kašnjenjem i gubitkom paketa.

7. Zaključak

U radu je projektiran i implementiran cjelovit sustav za udaljeni nadzor računala s operativnim sustavom Linux. Izrađena je agentska aplikacija za prikupljanje sistemskih pokazatelja, poslužiteljska aplikacija sa sučeljem REST i bazom podataka te web sučelje s grafičkim prikazom trenutačnih i povijesnih podataka. Ostvareni su i sustav upozorenja s podesivim pragovima i otkrivanjem nedostupnih računala te mogućnost udaljenog upravljanja procesima. Time su ispunjene sve funkcionalnosti predviđene zadatkom rada.

Provedenim je ispitivanjem potvrđeno da sustav ispravno prikuplja i prikazuje podatke, pravodobno generira upozorenja te zadržava stabilnost pri privremenom prekidu mrežne veze. Sustav je primjenjiv za nadzor manjega broja računala u okruženjima u kojima uvođenje složenijih nadzornih rješenja nije opravdano, primjerice u malim tvrtkama, laboratorijima ili kućnim poslužiteljskim okruženjima. Njegova modularna struktura omogućuje jednostavno proširivanje.

Moguća poboljšanja uključuju uvođenje šifrirane komunikacije protokolom HTTPS, slanje obavijesti o upozorenjima elektroničkom poštom te nadzor dodatnih pokazatelja poput temperature, pojedinačnih particija diska i pojedinačnih mrežnih sučelja. U području pohrane podataka prvi je korak uključivanje periodičnog čišćenja starijih pokazatelja funkcijom `prune_old_metrics`, koje je u sustavu pripremljeno, ali se još ne pokreće automatski; nakon toga slijedi sažimanje starijih podataka u prosjeke po satu ili danu, čime bi se povijest mogla čuvati dulje uz znatno manju datoteku baze podataka. Naposljetku, pakiranje agenta kao usluge `systemd` [8] pojednostavnilo bi instalaciju i omogućilo automatsko pokretanje pri podizanju sustava.

Literatura

[1] Zabbix LLC: „Zabbix Documentation”, https://www.zabbix.com/documentation/current/en (posjećeno 12.4.2026.)

[2] Nagios Enterprises: „Nagios Core Documentation”, https://www.nagios.org/documentation/ (posjećeno 12.4.2026.)

[3] Rodolà, G.: „psutil Documentation”, https://psutil.readthedocs.io/ (posjećeno 18.5.2026.)

[4] Pallets Projects: „Flask Documentation”, https://flask.palletsprojects.com/ (posjećeno 18.4.2026.)

[5] SQLite Consortium: „SQLite Documentation”, https://www.sqlite.org/docs.html (posjećeno 20.4.2026.)

[6] Chart.js Contributors: „Chart.js Documentation”, https://www.chartjs.org/docs/ (posjećeno 10.5.2026.)

[7] Moriarty, K., Kaliski, B., Rusch, A.: „PKCS #5: Password-Based Cryptography Specification Version 2.1”, RFC 8018, Internet Engineering Task Force, 2017.

[8] freedesktop.org: „systemd System and Service Manager Documentation”, https://www.freedesktop.org/wiki/Software/systemd/ (posjećeno 15.7.2026.)

Popis slika

Slika 1: Arhitektura sustava 6

Slika 2: Dijagram E-R baze podataka 8

Slika 3: Slijed razmjene poruka između agenta i poslužitelja 10

Slika 4: Stranica za prijavu korisnika 24

Slika 5: Nadzorna ploča s pregledom nadziranih računala 25

Slika 6: Detaljni prikaz računala s grafovima pokazatelja 25

Slika 7: Popis procesa nadziranog računala 26

Slika 8: Pregled generiranih upozorenja 27

Slika 9: Stranica za uređivanje pragova upozorenja 27

Slika 10: Upozorenja generirana tijekom ispitivanja opterećenja 30

Popis tablica

Tablica 1: Najvažnije krajnje točke sučelja REST 7

Tablica 2: Tablice baze podataka 8

Tablica 3: Sastav ispitnog okruženja 29

Tablica 4: Pregled provedenih ispitivanja i rezultata 31

Popis ispisa

Ispis 1: Konfiguracijska datoteka agent_config.json 13

Ispis 2: Metoda za prikupljanje sistemskih pokazatelja 14

Ispis 3: Glavna petlja agentske aplikacije 15

Ispis 4: Krajnja točka za zaprimanje pokazatelja 16

Ispis 5: Definicija tablice metrics i pripadajućega indeksa 17

Ispis 6: Provjera pragova i generiranje upozorenja 19

Ispis 7: Pozadinska dretva za otkrivanje nedostupnosti 20

Ispis 8: Izvadak iz skupa automatiziranih provjera 32

Dodatak

U prilogu je dan cjeloviti programski kôd obiju komponenti sustava: agentske aplikacije, četiriju modula poslužiteljske aplikacije i skupa automatiziranih provjera opisanoga u potpoglavlju 6.7. Automatizirano ispitivanje. Kôd je dodatno dostupan u udaljenom repozitoriju sustava za upravljanje verzijama Git.

Prilog 1: Agentska aplikacija – agent.py

```
import json
import signal as signal_module
import socket
import subprocess
import sys
import time
from pathlib import Path

import psutil
import requests


CONFIG_PATH = Path(__file__).with_name("agent_config.json")

DEFAULT_CONFIG = {
    "server_url": "http://192.168.1.76:5000",
    "interval":   5,
    "hostname":   socket.gethostname(),
    "description": "",
    "api_key":    None,
    "host_id":    None,
    "allow_remote_commands": True,
    "max_processes_in_list": 200,
}


# ---------- config ----------

def load_config() -> dict:
    if CONFIG_PATH.exists():
        cfg = DEFAULT_CONFIG.copy()
        cfg.update(json.loads(CONFIG_PATH.read_text()))
        return cfg
    save_config(DEFAULT_CONFIG)
    return DEFAULT_CONFIG.copy()


def save_config(cfg: dict):
    CONFIG_PATH.write_text(json.dumps(cfg, indent=2))
```

```
def register(cfg: dict) -> dict:
    """Register this host with the server and obtain an API key."""
    url = cfg["server_url"].rstrip("/") + "/api/register"
    payload = {"hostname": cfg["hostname"], "description": cfg.get("description",
"")}
    r = requests.post(url, json=payload, timeout=10)
    r.raise_for_status()
    data = r.json()
    cfg["api_key"]  = data["api_key"]
    cfg["host_id"]  = data["host_id"]
    cfg["interval"] = data.get("polling_interval", cfg["interval"])
    save_config(cfg)
    print(f"[agent] Registered as host_id={cfg['host_id']}, key stored.")
    return cfg



def cmd_list_processes(payload: dict, cfg: dict) -> dict:
    """Return a list of processes sorted by CPU% descending."""
    max_n = int(cfg.get("max_processes_in_list", 200))
    procs = []

    for p in psutil.process_iter(["pid"]):
        try:
            p.cpu_percent(interval=None)
        except (psutil.NoSuchProcess, psutil.AccessDenied):
            pass
    time.sleep(0.3)

    for p in psutil.process_iter(["pid", "name", "username", "memory_percent",
"cmdline"]):
        try:
            info = p.info
            cpu  = p.cpu_percent(interval=None)
            cmdline = info.get("cmdline") or []
            procs.append({
                "pid":     info["pid"],
                "name":    info.get("name") or "?",
                "user":    info.get("username") or "?",
                "cpu":     round(cpu, 1),
                "mem":     round(info.get("memory_percent") or 0, 1),
                "cmdline": " ".join(cmdline)[:300] if cmdline else "",
            })
        except (psutil.NoSuchProcess, psutil.AccessDenied):
            continue

    procs.sort(key=lambda x: x["cpu"], reverse=True)
    procs = procs[:max_n]
    return {"processes": procs, "total_returned": len(procs)}


def cmd_kill_process(payload: dict, cfg: dict) -> dict:
    """Send SIGTERM or SIGKILL to a process by PID."""
    pid     = int(payload.get("pid"))
    sig     = (payload.get("signal") or "TERM").upper()
```

```
    sig_num = signal_module.SIGKILL if sig == "KILL" else signal_module.SIGTERM
    try:
        proc = psutil.Process(pid)
        name = proc.name()
        proc.send_signal(sig_num)
        try:
            proc.wait(timeout=3)
            gone = True
        except psutil.TimeoutExpired:
            gone = not proc.is_running()
        return {"killed": True, "pid": pid, "name": name, "signal": sig,
                "gone": gone,
                "message": f"sent SIG{sig} to {name} (PID {pid})"
                           + ("" if gone else "; still running after 3s")}
    except psutil.NoSuchProcess:
        return {"killed": False, "pid": pid, "message": f"PID {pid} does not
exist"}
    except psutil.AccessDenied:
        return {"killed": False, "pid": pid,
                "message": f"Access denied for PID {pid} (agent lacks sufficient
privileges)"}


def cmd_exec(payload: dict, cfg: dict) -> tuple:
    """Run a shell command and return (result_dict, exit_code)."""
    cmd     = payload.get("cmd", "")
    timeout = int(payload.get("timeout", 30))
    try:
        proc = subprocess.run(
            ["bash", "-c", cmd],
            capture_output=True, text=True, timeout=timeout,
        )
        return ({
            "stdout":    proc.stdout,
            "stderr":    proc.stderr,
            "exit_code": proc.returncode,
            "timeout":   False,
        }, proc.returncode)
    except subprocess.TimeoutExpired as e:
        return ({
            "stdout":    e.stdout.decode() if isinstance(e.stdout, bytes) else
(e.stdout or ""),
            "stderr":    e.stderr.decode() if isinstance(e.stderr, bytes) else
(e.stderr or ""),
            "exit_code": None,
            "timeout":   True,
            "error":     f"timeout after {timeout}s",
        }, -1)


COMMAND_HANDLERS = {
    "list_processes": cmd_list_processes,
    "kill_process":   cmd_kill_process,
}
```

```
def handle_command(cmd: dict, cfg: dict, base_url: str, headers: dict):
    """Execute a single command and post the result back to the server."""
    cmd_id   = cmd["id"]
    cmd_type = cmd["type"]
    payload  = cmd.get("payload") or {}
    print(f"[cmd #{cmd_id}] {cmd_type} {payload}")

    try:
        if cmd_type == "exec":
            result, exit_code = cmd_exec(payload, cfg)
            status = "done" if not result.get("timeout") else "failed"
            error  = result.get("error") if result.get("timeout") else None
            body = {"status": status, "exit_code": exit_code,
                    "result": result, "error": error}
        elif cmd_type in COMMAND_HANDLERS:
            handler = COMMAND_HANDLERS[cmd_type]
            result = handler(payload, cfg)
            body = {"status": "done", "exit_code": 0, "result": result}
        else:
            body = {"status": "failed", "error": f"unknown command type:
{cmd_type}"}
    except Exception as e:
        body = {"status": "failed", "error": f"{type(e).__name__}: {e}"}

    try:
        r = requests.post(
            base_url + f"/api/commands/{cmd_id}/result",
            json=body, headers=headers, timeout=15,
        )
        if not r.ok:
            print(f"[cmd #{cmd_id}] server rejected the result: {r.status_code}
{r.text[:200]}")
    except requests.RequestException as e:
        print(f"[cmd #{cmd_id}] error sending result: {e}")



class MetricsCollector:
    """Encapsulates state needed for delta computations (network counters)."""

    def __init__(self):
        self._prev_net  = psutil.net_io_counters()
        self._prev_time = time.time()
        psutil.cpu_percent(interval=None)

    def collect(self) -> dict:
        now = time.time()

        cpu_pct = psutil.cpu_percent(interval=None)
        vm      = psutil.virtual_memory()
        disk    = psutil.disk_usage("/")
        proc_n  = len(psutil.pids())
        uptime  = int(now - psutil.boot_time())

        net      = psutil.net_io_counters()
        elapsed  = max(now - self._prev_time, 0.001)
```

```
        sent_kbs = (net.bytes_sent - self._prev_net.bytes_sent) / 1024.0 /
elapsed
        recv_kbs = (net.bytes_recv - self._prev_net.bytes_recv) / 1024.0 /
elapsed
        self._prev_net  = net
        self._prev_time = now

        return {
            "cpu":       round(cpu_pct, 2),
            "ram":       round(vm.percent, 2),
            "disk":      round(disk.percent, 2),
            "net_sent":  round(max(sent_kbs, 0), 2),
            "net_recv":  round(max(recv_kbs, 0), 2),
            "processes": proc_n,
            "uptime":    uptime,
        }



def main():
    cfg = load_config()
    print(f"[agent] hostname = {cfg['hostname']}")
    print(f"[agent] server   = {cfg['server_url']}")

    if not cfg.get("api_key"):
        print("[agent] No API key, registering...")
        try:
            cfg = register(cfg)
        except Exception as e:
            print(f"[agent] Registration failed: {e}")
            sys.exit(1)

    metrics_url = cfg["server_url"].rstrip("/") + "/api/metrics"
    headers     = {"X-API-Key": cfg["api_key"]}
    interval    = max(int(cfg.get("interval", 5)), 1)
    collector   = MetricsCollector()

    time.sleep(1)

    while True:
        loop_start = time.time()
        try:
            data = collector.collect()
            r = requests.post(metrics_url, json=data, headers=headers,
timeout=10)
            if r.status_code == 401:
                print("[agent] API key rejected, attempting to re-register...")
                cfg = register(cfg)
                headers = {"X-API-Key": cfg["api_key"]}
            elif r.ok:
                resp = r.json()
                new_interval = resp.get("polling_interval")
                if new_interval and new_interval != interval:
                    interval = max(int(new_interval), 1)
                    cfg["interval"] = interval
                    save_config(cfg)
```

```
                    print(f"[agent] Interval changed to {interval}s.")
                print(f"[sent] cpu={data['cpu']}% ram={data['ram']}% "
                      f"disk={data['disk']}% proc={data['processes']} "
                      f"net=(down {data['net_recv']}KB/s up
{data['net_sent']}KB/s) "
                      f"up={data['uptime']}s")

                pending = resp.get("commands") or []
                if pending:
                    if cfg.get("allow_remote_commands", True):
                        print(f"[agent] {len(pending)} new command(s) to
execute")
                        for c in pending:
                            handle_command(c, cfg,
                                           cfg["server_url"].rstrip("/"),
headers)
                    else:
                        print(f"[agent] {len(pending)} command(s) refused
(allow_remote_commands=false)")
                        for c in pending:
                            try:
                                requests.post(
                                    cfg["server_url"].rstrip("/") +
f"/api/commands/{c['id']}/result",
                                    json={"status": "failed",
                                          "error": "allow_remote_commands is
disabled in agent_config.json"},
                                    headers=headers, timeout=10,
                                )
                            except requests.RequestException:
                                pass
            else:
                print(f"[agent] Server returned {r.status_code}: {r.text[:200]}")
        except requests.RequestException as e:
            print(f"[agent] Network error: {e}")
        except Exception as e:
            print(f"[agent] Error: {e}")

        elapsed = time.time() - loop_start
        time.sleep(max(interval - elapsed, 0))


if __name__ == "__main__":
    main()
```

Prilog 2: Poslužiteljska aplikacija – server.py

```
import os
import secrets
from functools import wraps

from flask import (Flask, request, jsonify, render_template, redirect,
                   url_for, session, flash, abort)

import database as db
```

```
import alerts as alert_engine
from services import (HostService, MetricsService, ThresholdService,
                      AlertService, CommandService, ValidationError)


app = Flask(__name__)
app.secret_key = os.environ.get("MONITOR_SECRET", secrets.token_hex(32))




def login_required(fn):
    @wraps(fn)
    def wrapper(*args, **kwargs):
        if "user_id" not in session:
            return redirect(url_for("login", next=request.path))
        return fn(*args, **kwargs)
    return wrapper


def get_api_key():
    """Read the API key from a header or the JSON body."""
    key = request.headers.get("X-API-Key")
    if not key and request.is_json:
        key = (request.get_json(silent=True) or {}).get("api_key")
    return key


def authenticated_host():
    """Resolve the host that owns the request's API key, or return an error
    tuple suitable for `return`ing directly from a route."""
    key = get_api_key()
    if not key:
        return None, (jsonify({"error": "missing api key"}), 401)
    host = db.get_host_by_api_key(key)
    if not host:
        return None, (jsonify({"error": "invalid api key"}), 401)
    return host, None


def format_uptime(seconds):
    seconds = int(seconds or 0)
    d, rem = divmod(seconds, 86400)
    h, rem = divmod(rem, 3600)
    m, _   = divmod(rem, 60)
    if d:  return f"{d}d {h}h {m}min"
    if h:  return f"{h}h {m}min"
    return f"{m}min"


app.jinja_env.filters["uptime"] = format_uptime




@app.route("/api/register", methods=["POST"])
```

```
def api_register():
    """First call from an agent. Sends { 'hostname', 'description' } and
    receives an API key used for subsequent authentication."""
    data = request.get_json(silent=True) or {}
    try:
        host = HostService.register(data.get("hostname", ""),
data.get("description", ""))
    except ValidationError as e:
        return jsonify({"error": str(e)}), 400
    return jsonify({
        "host_id": host["id"],
        "hostname": host["hostname"],
        "api_key": host["api_key"],
        "polling_interval": host["polling_interval"],
    }), 201


@app.route("/api/metrics", methods=["POST"])
def api_metrics():
    """Agent submits metrics (auth via X-API-Key). The response carries the
    current polling interval and any commands waiting for the agent."""
    host, err = authenticated_host()
    if err:
        return err
    if not host["active"]:
        return jsonify({"error": "host is disabled"}), 403

    try:
        MetricsService.ingest(host, request.get_json(silent=True) or {})
    except ValidationError as e:
        return jsonify({"error": str(e)}), 400

    return jsonify({
        "status": "ok",
        "polling_interval": host["polling_interval"],
        "active": bool(host["active"]),
        "commands": CommandService.pending_for_agent(host["id"]),
    })


@app.route("/api/commands/<int:cmd_id>/result", methods=["POST"])
def api_command_result(cmd_id):
    """Agent submits the result of an executed command."""
    host, err = authenticated_host()
    if err:
        return err

    try:
        CommandService.authorize_result(cmd_id, host)
    except LookupError:
        return jsonify({"error": "command not found"}), 404
    except PermissionError:
        return jsonify({"error": "command does not belong to this host"}), 403

    data = request.get_json(silent=True) or {}
    CommandService.save_result(
```

```
        cmd_id,
        status=data.get("status", "done"),
        exit_code=data.get("exit_code"),
        result=data.get("result"),
        error=data.get("error"),
    )
    return jsonify({"status": "ok"})


@app.route("/api/hosts", methods=["GET"])
@login_required
def api_list_hosts():
    return jsonify(HostService.list_with_status())


@app.route("/api/hosts/<int:host_id>/metrics", methods=["GET"])
@login_required
def api_host_metrics(host_id):
    """Query params: range = 1h|6h|24h|7d (default 1h), limit = int (default
500)."""
    if not HostService.get(host_id):
        abort(404)
    return jsonify(MetricsService.history(
        host_id,
        range_str=request.args.get("range", "1h"),
        limit=request.args.get("limit", 500),
    ))


@app.route("/api/alerts", methods=["GET"])
@login_required
def api_alerts():
    return jsonify(AlertService.list(
        host_id=request.args.get("host_id", type=int),
        only_unresolved=request.args.get("only_unresolved", "false").lower() ==
"true",
    ))




@app.route("/login", methods=["GET", "POST"])
def login():
    if request.method == "POST":
        user = db.authenticate(request.form.get("username", ""),
                               request.form.get("password", ""))
        if user:
            session["user_id"] = user["id"]
            session["username"] = user["username"]
            return redirect(request.args.get("next") or url_for("dashboard"))
        flash("Invalid username or password.", "error")
    return render_template("login.html")


@app.route("/logout")
def logout():
```

```
    session.clear()
    return redirect(url_for("login"))



@app.route("/")
@login_required
def dashboard():
    return render_template("dashboard.html",
                           hosts=HostService.list_with_status(),
                           alerts=AlertService.list(only_unresolved=True,
limit=10))


@app.route("/hosts/<int:host_id>")
@login_required
def host_detail(host_id):
    host = HostService.get(host_id)
    if not host:
        abort(404)
    return render_template("host_detail.html", host=host,
                           range=request.args.get("range", "1h"))


@app.route("/hosts/<int:host_id>/processes", methods=["GET", "POST"])
@login_required
def host_processes(host_id):
    """Processes: refresh the list, kill by PID, run an arbitrary shell
command."""
    host = HostService.get(host_id)
    if not host:
        abort(404)

    if request.method == "POST":
        action = request.form.get("action")
        username = session.get("username", "unknown")
        try:
            if action == "refresh":
                queued = CommandService.request_process_list(host_id, username)
                flash("Process list requested. It will appear after the agent's
next poll."
                      if queued else "A refresh is already pending.", "info")
            elif action == "kill":
                CommandService.request_kill(host_id, request.form.get("pid"),
                                            request.form.get("signal", "TERM"),
username)
                flash(f"Requested kill of PID={request.form.get('pid')} "
                      f"({request.form.get('signal', 'TERM')}).", "info")
            elif action == "exec":
                CommandService.request_exec(host_id, request.form.get("cmd", ""),
                                            request.form.get("timeout", 30),
username)
                flash("Command queued.", "info")
        except ValidationError as e:
            flash(str(e), "error")
        return redirect(url_for("host_processes", host_id=host_id))
```

```
    processes, last_refresh = CommandService.process_view(host_id,
request.args.get("q", ""))
    return render_template("processes.html",
                           host=host, processes=processes,
                           last_refresh=last_refresh, q=request.args.get("q",
"").strip(),
                           history=CommandService.history(host_id, limit=15))


@app.route("/hosts", methods=["GET", "POST"])
@login_required
def hosts_manage():
    """Manual host addition (generates an API key) and editing of settings."""
    if request.method == "POST":
        action = request.form.get("action")
        try:
            if action == "add":
                h = HostService.register(request.form.get("hostname", ""),
                                         request.form.get("description", ""))
                flash(f"Host '{h['hostname']}' added. API key: {h['api_key']}",
"success")
            elif action == "update":
                HostService.update_config(
                    int(request.form["host_id"]),
                    polling_interval=request.form.get("polling_interval", 5),
                    active=1 if request.form.get("active") else 0,
                    description=request.form.get("description", ""),
                )
                flash("Settings saved.", "success")
            elif action == "delete":
                HostService.delete(int(request.form["host_id"]))
                flash("Host deleted.", "success")
        except ValidationError as e:
            flash(str(e), "error")
        return redirect(url_for("hosts_manage"))

    return render_template("hosts.html", hosts=HostService.list_with_status())


@app.route("/alerts")
@login_required
def alerts_page():
    only_unresolved = request.args.get("only_unresolved", "false").lower() ==
"true"
    return render_template("alerts.html",

alerts=AlertService.list(only_unresolved=only_unresolved),
                           only_unresolved=only_unresolved)


@app.route("/alerts/<int:alert_id>/resolve", methods=["POST"])
@login_required
def resolve(alert_id):
    AlertService.resolve(alert_id)
    return redirect(request.referrer or url_for("alerts_page"))
```

```
@app.route("/settings", methods=["GET", "POST"])
@login_required
def settings():
    """Global alert thresholds."""
    if request.method == "POST":
        try:
            for metric in ("cpu", "ram", "disk"):
                ThresholdService.save(
                    metric,
                    warning=request.form.get(f"{metric}_warning", 80),
                    critical=request.form.get(f"{metric}_critical", 95),
                    host_id=None,
                )
            flash("Thresholds saved.", "success")
        except ValidationError as e:
            flash(str(e), "error")
        return redirect(url_for("settings"))
    return render_template("settings.html",
thresholds=ThresholdService.get(None))



def create_app():
    db.init_db()
    alert_engine.start_background_monitor(interval_s=30)
    return app


if __name__ == "__main__":
    create_app()
    print("=" * 60)
    print(" Linux Remote Monitoring System")
    print("=" * 60)
    print(f" Database: {db.DB_PATH}")
    print(" Default login: admin / admin  (please change after first sign-in)")
    print(" Listening on http://0.0.0.0:5000")
    print("=" * 60)
    app.run(host="0.0.0.0", port=5000, debug=False)
```

Prilog 3: Poslužiteljska aplikacija – services.py

```
from datetime import datetime, timedelta

import database as db




MIN_POLLING_INTERVAL = 1
MAX_POLLING_INTERVAL = 3600
```

```
PCT_MIN = 0.0
PCT_MAX = 100.0

ALLOWED_COMMAND_TYPES = ("list_processes", "kill_process", "exec")
ALLOWED_KILL_SIGNALS = ("TERM", "KILL")
MIN_EXEC_TIMEOUT = 1
MAX_EXEC_TIMEOUT = 600

RANGE_TO_HOURS = {"1h": 1, "6h": 6, "24h": 24, "7d": 24 * 7}
DEFAULT_RANGE = "1h"
MAX_METRIC_POINTS = 5000

ONLINE_MIN_SECONDS = 15
OFFLINE_MIN_SECONDS = 30
OFFLINE_MULTIPLIER = 3


class ValidationError(ValueError):
    """Raised when input violates a business rule. The routes catch this and
    turn it into a flash message or an HTTP 400 response."""


def _clamp(value, low, high):
    return max(low, min(high, value))


def _coerce_float(value, field):
    try:
        return float(value)
    except (TypeError, ValueError):
        raise ValidationError(f"{field} must be a number")


def _coerce_int(value, field):
    try:
        return int(value)
    except (TypeError, ValueError):
        raise ValidationError(f"{field} must be a whole number")




class HostService:
    """Domain rules about monitored machines: registration, online status and
    configuration limits."""

    @staticmethod
    def register(hostname, description=""):
        hostname = (hostname or "").strip()
        if not hostname:
            raise ValidationError("hostname is required")
        if len(hostname) > 255:
            raise ValidationError("hostname is too long (max 255 characters)")
        return db.register_host(hostname, (description or "").strip())
```

```
    @staticmethod
    def is_online(host):
        """A host is online if its most recent report is newer than
        OFFLINE_MULTIPLIER * polling_interval seconds, with a floor so very
        short intervals do not flap between states."""
        last_seen = host.get("last_seen")
        if not last_seen:
            return False
        window = max(ONLINE_MIN_SECONDS, host["polling_interval"] *
OFFLINE_MULTIPLIER)
        age = (datetime.utcnow() -
datetime.fromisoformat(last_seen)).total_seconds()
        return age < window

    @staticmethod
    def list_with_status():
        """Every host plus its latest metric and a computed online flag."""
        hosts = db.list_hosts()
        for h in hosts:
            h["online"] = HostService.is_online(h)
        return hosts

    @staticmethod
    def get(host_id):
        return db.get_host(host_id)

    @staticmethod
    def update_config(host_id, polling_interval=None, active=None,
description=None):
        if polling_interval is not None:
            polling_interval = _clamp(
                _coerce_int(polling_interval, "polling_interval"),
                MIN_POLLING_INTERVAL, MAX_POLLING_INTERVAL,
            )
        db.update_host_config(
            host_id,
            polling_interval=polling_interval,
            active=active,
            description=description,
        )

    @staticmethod
    def delete(host_id):
        db.delete_host(host_id)




class MetricsService:
    """Rules for accepting, normalising and querying time-series metrics."""

    @staticmethod
    def normalize(payload):
        """Turn a raw agent payload into a clean record. Percentages are clamped
        to 0..100; counters that can never be negative are floored at 0. This
        protects the database and the charts from a misbehaving or hostile
```

```
        agent."""
        payload = payload or {}
        return {
            "cpu":       _clamp(_coerce_float(payload.get("cpu", 0), "cpu"),
PCT_MIN, PCT_MAX),
            "ram":       _clamp(_coerce_float(payload.get("ram", 0), "ram"),
PCT_MIN, PCT_MAX),
            "disk":      _clamp(_coerce_float(payload.get("disk", 0), "disk"),
PCT_MIN, PCT_MAX),
            "net_sent":  max(_coerce_float(payload.get("net_sent", 0),
"net_sent"), 0.0),
            "net_recv":  max(_coerce_float(payload.get("net_recv", 0),
"net_recv"), 0.0),
            "processes": max(_coerce_int(payload.get("processes", 0),
"processes"), 0),
            "uptime":    max(_coerce_int(payload.get("uptime", 0), "uptime"), 0),
        }

    @staticmethod
    def ingest(host, payload):
        """Full ingestion pipeline for one metric submission: normalise, store,
        update the host's last-seen timestamp and evaluate alerts."""
        record = MetricsService.normalize(payload)
        db.insert_metric(host["id"], record)
        db.update_host_seen(host["id"])
        AlertService.evaluate_metric(host["id"], host["hostname"], record)
        return record

    @staticmethod
    def range_to_since(range_str):
        """Map a UI range token (1h, 6h, 24h, 7d) to an absolute UTC cutoff."""
        hours = RANGE_TO_HOURS.get(range_str, RANGE_TO_HOURS[DEFAULT_RANGE])
        return datetime.utcnow() - timedelta(hours=hours)

    @staticmethod
    def history(host_id, range_str=DEFAULT_RANGE, limit=500):
        limit = _clamp(_coerce_int(limit, "limit"), 1, MAX_METRIC_POINTS)
        since = MetricsService.range_to_since(range_str)
        return db.get_metrics(host_id, since=since, limit=limit)




class ThresholdService:
    """Validation and retrieval of alert thresholds."""

    @staticmethod
    def get(host_id=None):
        return db.get_thresholds(host_id)

    @staticmethod
    def save(metric, warning, critical, host_id=None):
        warning = _clamp(_coerce_float(warning, f"{metric} warning"), PCT_MIN,
PCT_MAX)
        critical = _clamp(_coerce_float(critical, f"{metric} critical"), PCT_MIN,
PCT_MAX)
```

```
        if warning >= critical:
            raise ValidationError(
                f"{metric.upper()}: warning ({warning:.0f}%) must be lower than "
                f"critical ({critical:.0f}%)"
            )
        db.set_threshold(metric, warning, critical, host_id=host_id)


class AlertService:
    """The alerting policy: deciding when a metric or an absent host warrants an
    alert. The actual storage and de-duplication are handled by database.py;
    the decision of warning-vs-critical-vs-nothing is made here."""

    METRIC_LABELS = {"cpu": "CPU usage", "ram": "RAM usage", "disk": "Disk
usage"}

    @staticmethod
    def evaluate_metric(host_id, hostname, record):
        """Compare a freshly stored metric against the effective thresholds and
        raise warning or critical alerts where the value crosses a limit."""
        thresholds = db.get_thresholds(host_id)
        for metric, label in AlertService.METRIC_LABELS.items():
            t = thresholds.get(metric)
            if not t:
                continue
            value = record.get(metric, 0)
            level = AlertService._level_for(value, t)
            if level is None:
                continue
            limit = t[level]
            db.insert_alert(
                host_id, metric, level,
                f"{label} on {hostname} = {value:.1f}% ({level} >= {limit}%)",
                value=value, threshold=limit,
            )

    @staticmethod
    def _level_for(value, threshold):
        if value >= threshold["critical"]:
            return "critical"
        if value >= threshold["warning"]:
            return "warning"
        return None

    @staticmethod
    def check_offline_hosts():
        """Raise a critical alert for any active host that has gone silent for
        longer than its offline window."""
        now = datetime.utcnow()
        for h in db.list_hosts():
            if not h["active"] or not h["last_seen"]:
                continue
            window = max(OFFLINE_MIN_SECONDS, h["polling_interval"] *
OFFLINE_MULTIPLIER)
            silent_for = (now -
datetime.fromisoformat(h["last_seen"])).total_seconds()
```

```
            if silent_for > window:
                db.insert_alert(
                    h["id"], "offline", "critical",
                    f"Host {h['hostname']} is unreachable (last seen
{h['last_seen']} UTC)",
                )

    @staticmethod
    def list(host_id=None, only_unresolved=False, limit=200):
        return db.list_alerts(host_id=host_id, only_unresolved=only_unresolved,
limit=limit)

    @staticmethod
    def resolve(alert_id):
        db.resolve_alert(alert_id)




class CommandService:
    """The rules around queuing remote commands for an agent: what is allowed,
    how arguments are validated, and how results are authorised when they come
    back."""

    @staticmethod
    def request_process_list(host_id, issued_by):
        """Queue a list_processes command, but only if one is not already
        waiting. Without this guard the dashboard's auto-refresh could pile up
        dozens of identical requests faster than the agent can answer them."""
        for c in db.list_commands(host_id, limit=5, types=["list_processes"]):
            if c["status"] in ("pending", "executing"):
                return None
        return db.queue_command(host_id, "list_processes", {}, issued_by)

    @staticmethod
    def request_kill(host_id, pid, signal, issued_by):
        pid = _coerce_int(pid, "pid")
        if pid <= 1:
            raise ValidationError("refusing to signal PID <= 1 (init / kernel)")
        signal = (signal or "TERM").upper()
        if signal not in ALLOWED_KILL_SIGNALS:
            raise ValidationError(f"signal must be one of
{ALLOWED_KILL_SIGNALS}")
        return db.queue_command(host_id, "kill_process",
                                {"pid": pid, "signal": signal}, issued_by)

    @staticmethod
    def request_exec(host_id, cmd, timeout, issued_by):
        cmd = (cmd or "").strip()
        if not cmd:
            raise ValidationError("command is empty")
        timeout = _clamp(_coerce_int(timeout, "timeout"), MIN_EXEC_TIMEOUT,
MAX_EXEC_TIMEOUT)
        return db.queue_command(host_id, "exec",
                                {"cmd": cmd, "timeout": timeout}, issued_by)
```

```
    @staticmethod
    def authorize_result(cmd_id, host):
        """Return the command if it exists and belongs to this host, otherwise
        raise so the route can answer with the right HTTP status."""
        cmd = db.get_command(cmd_id)
        if not cmd:
            raise LookupError("command not found")
        if cmd["host_id"] != host["id"]:
            raise PermissionError("command does not belong to this host")
        return cmd

    @staticmethod
    def save_result(cmd_id, status, exit_code=None, result=None, error=None):
        db.save_command_result(cmd_id, status=status, exit_code=exit_code,
                               result=result, error=error)

    @staticmethod
    def pending_for_agent(host_id):
        """Hand the agent its pending commands in the compact shape it expects,
        marking them as executing so they are not handed out twice."""
        pending = db.get_pending_commands(host_id, mark_executing=True)
        return [{"id": c["id"], "type": c["type"], "payload": c["payload"]} for c
in pending]

    @staticmethod
    def process_view(host_id, query=""):
        """Latest process list for a host with an optional case-insensitive
        filter applied across pid, name, user and cmdline."""
        db.cleanup_stuck_commands(stale_minutes=5)
        latest = db.get_latest_process_list(host_id)
        processes, last_refresh = [], None
        if latest and latest["result"]:
            processes = latest["result"].get("processes", [])
            last_refresh = latest["executed_at"]

        query = (query or "").strip().lower()
        if query:
            processes = [
                p for p in processes
                if query in str(p.get("pid"))
                or query in (p.get("name") or "").lower()
                or query in (p.get("user") or "").lower()
                or query in (p.get("cmdline") or "").lower()
            ]
        return processes, last_refresh

    @staticmethod
    def history(host_id, limit=15):
        return db.list_commands(host_id, limit=limit)
```

Prilog 4: Poslužiteljska aplikacija – database.py

```
"""
database.py
-----------
```

```
SQLite layer for the Linux remote monitoring system.

Tables:
  hosts       - monitored machines (hostname, API key, polling interval, ...)
  metrics     - time-series of metrics per host
  alerts      - generated alerts
  thresholds  - alert thresholds (global or per-host)
  users       - web UI users
  commands    - remote command queue (web UI inserts, agent pulls and reports
back)
"""

import sqlite3
import secrets
import hashlib
import os
import json as _json
from datetime import datetime, timedelta
from contextlib import contextmanager

DB_PATH = os.path.join(os.path.dirname(__file__), "monitoring.db")


# ---------- low level ----------

@contextmanager
def get_conn():
    """Yields a connection with row_factory and foreign keys enabled."""
    conn = sqlite3.connect(DB_PATH)
    conn.row_factory = sqlite3.Row
    conn.execute("PRAGMA foreign_keys = ON")
    try:
        yield conn
        conn.commit()
    finally:
        conn.close()


def init_db():
    """Create tables and seed default values if the database is empty."""
    with get_conn() as c:
        c.executescript("""
            CREATE TABLE IF NOT EXISTS hosts (
                id               INTEGER PRIMARY KEY AUTOINCREMENT,
                hostname         TEXT NOT NULL UNIQUE,
                api_key          TEXT NOT NULL UNIQUE,
                registered_at    TEXT NOT NULL,
                last_seen        TEXT,
                polling_interval INTEGER NOT NULL DEFAULT 5,
                active           INTEGER NOT NULL DEFAULT 1,
                description      TEXT DEFAULT ''
            );

            CREATE TABLE IF NOT EXISTS metrics (
                id         INTEGER PRIMARY KEY AUTOINCREMENT,
                host_id    INTEGER NOT NULL,
```

```
                ts          TEXT NOT NULL,
                cpu         REAL NOT NULL,
                ram         REAL NOT NULL,
                disk        REAL NOT NULL,
                net_sent    REAL NOT NULL DEFAULT 0,    -- KB/s during interval
                net_recv    REAL NOT NULL DEFAULT 0,    -- KB/s during interval
                processes   INTEGER NOT NULL DEFAULT 0,
                uptime      INTEGER NOT NULL DEFAULT 0, -- seconds since boot
                FOREIGN KEY (host_id) REFERENCES hosts(id) ON DELETE CASCADE
            );
            CREATE INDEX IF NOT EXISTS idx_metrics_host_ts ON metrics(host_id,
ts);

            CREATE TABLE IF NOT EXISTS alerts (
                id          INTEGER PRIMARY KEY AUTOINCREMENT,
                host_id     INTEGER NOT NULL,
                ts          TEXT NOT NULL,
                metric      TEXT NOT NULL,      -- cpu, ram, disk, offline
                level       TEXT NOT NULL,      -- warning, critical
                value       REAL,
                threshold   REAL,
                message     TEXT NOT NULL,
                resolved    INTEGER NOT NULL DEFAULT 0,
                FOREIGN KEY (host_id) REFERENCES hosts(id) ON DELETE CASCADE
            );
            CREATE INDEX IF NOT EXISTS idx_alerts_host_ts ON alerts(host_id, ts);

            CREATE TABLE IF NOT EXISTS thresholds (
                id      INTEGER PRIMARY KEY AUTOINCREMENT,
                host_id INTEGER,            -- NULL = global threshold
                metric  TEXT NOT NULL,      -- cpu, ram, disk
                warning REAL NOT NULL,
                critical REAL NOT NULL,
                FOREIGN KEY (host_id) REFERENCES hosts(id) ON DELETE CASCADE
            );
            -- SQLite treats NULL != NULL inside UNIQUE constraints, so a plain
            -- UNIQUE(host_id, metric) would not prevent duplicate global rows.
            -- Two partial unique indexes solve this cleanly:
            CREATE UNIQUE INDEX IF NOT EXISTS idx_thr_global
                ON thresholds(metric) WHERE host_id IS NULL;
            CREATE UNIQUE INDEX IF NOT EXISTS idx_thr_host
                ON thresholds(host_id, metric) WHERE host_id IS NOT NULL;

            CREATE TABLE IF NOT EXISTS users (
                id           INTEGER PRIMARY KEY AUTOINCREMENT,
                username     TEXT NOT NULL UNIQUE,
                password_hash TEXT NOT NULL,
                created_at   TEXT NOT NULL
            );

            CREATE TABLE IF NOT EXISTS commands (
                id          INTEGER PRIMARY KEY AUTOINCREMENT,
                host_id     INTEGER NOT NULL,
                type        TEXT NOT NULL,        -- list_processes |
kill_process | exec
                payload     TEXT NOT NULL,        -- JSON string
```

```
                status      TEXT NOT NULL DEFAULT 'pending', --
pending|executing|done|failed
                created_at  TEXT NOT NULL,
                executed_at TEXT,
                exit_code   INTEGER,
                result      TEXT,                   -- JSON string
                error       TEXT,
                issued_by   TEXT,                   -- username that queued the
command
                FOREIGN KEY (host_id) REFERENCES hosts(id) ON DELETE CASCADE
            );
            CREATE INDEX IF NOT EXISTS idx_cmd_host_status ON commands(host_id,
status);
            CREATE INDEX IF NOT EXISTS idx_cmd_host_created ON commands(host_id,
created_at);
        """)

        # default global thresholds (only if not already present)
        defaults = [
            ("cpu",  80.0, 95.0),
            ("ram",  80.0, 95.0),
            ("disk", 85.0, 95.0),
        ]
        for metric, warn, crit in defaults:
            existing = c.execute(
                "SELECT 1 FROM thresholds WHERE host_id IS NULL AND metric = ?",
                (metric,),
            ).fetchone()
            if not existing:
                c.execute(
                    """INSERT INTO thresholds (host_id, metric, warning,
critical)
                       VALUES (NULL, ?, ?, ?)""",
                    (metric, warn, crit),
                )

        # default admin/admin user if no users exist
        row = c.execute("SELECT COUNT(*) AS n FROM users").fetchone()
        if row["n"] == 0:
            create_user(c, "admin", "admin")


# ---------- users ----------

def _hash_password(password: str, salt: str = None) -> str:
    """Return 'salt$hash' for the given password (PBKDF2-SHA256)."""
    if salt is None:
        salt = secrets.token_hex(16)
    h = hashlib.pbkdf2_hmac("sha256", password.encode(), salt.encode(), 120_000)
    return f"{salt}${h.hex()}"


def _verify_password(password: str, stored: str) -> bool:
    try:
        salt, _ = stored.split("$", 1)
    except ValueError:
```

```
        return False
    return secrets.compare_digest(_hash_password(password, salt), stored)


def create_user(conn, username: str, password: str):
    conn.execute(
        "INSERT INTO users (username, password_hash, created_at) VALUES (?, ?,
?)",
        (username, _hash_password(password),
datetime.utcnow().isoformat(timespec="seconds")),
    )


def authenticate(username: str, password: str):
    with get_conn() as c:
        row = c.execute("SELECT * FROM users WHERE username = ?",
(username,)).fetchone()
        if row and _verify_password(password, row["password_hash"]):
            return dict(row)
        return None


# ---------- hosts ----------

def register_host(hostname: str, description: str = "") -> dict:
    """Create a new host with a fresh API key. If the hostname exists, rotate the
key."""
    api_key = secrets.token_urlsafe(32)
    now = datetime.utcnow().isoformat(timespec="seconds")
    with get_conn() as c:
        existing = c.execute("SELECT id FROM hosts WHERE hostname = ?",
(hostname,)).fetchone()
        if existing:
            c.execute("UPDATE hosts SET api_key = ?, active = 1 WHERE id = ?",
                      (api_key, existing["id"]))
            host_id = existing["id"]
        else:
            cur = c.execute(
                """INSERT INTO hosts (hostname, api_key, registered_at,
polling_interval, description)
                   VALUES (?, ?, ?, 5, ?)""",
                (hostname, api_key, now, description),
            )
            host_id = cur.lastrowid
        host = c.execute("SELECT * FROM hosts WHERE id = ?",
(host_id,)).fetchone()
        return dict(host)


def get_host_by_api_key(api_key: str):
    with get_conn() as c:
        row = c.execute("SELECT * FROM hosts WHERE api_key = ?",
(api_key,)).fetchone()
        return dict(row) if row else None
```

```
def get_host(host_id: int):
    with get_conn() as c:
        row = c.execute("SELECT * FROM hosts WHERE id = ?",
(host_id,)).fetchone()
        return dict(row) if row else None


def list_hosts() -> list:
    """Return all hosts together with their latest metric (or None).

    Note: the online/offline decision is a business rule and lives in
    services.HostService, not here. This function only returns stored data.
    """
    with get_conn() as c:
        hosts = c.execute("SELECT * FROM hosts ORDER BY hostname").fetchall()
        result = []
        for h in hosts:
            latest = c.execute(
                "SELECT * FROM metrics WHERE host_id = ? ORDER BY ts DESC LIMIT
1",
                (h["id"],),
            ).fetchone()
            d = dict(h)
            d["latest"] = dict(latest) if latest else None
            result.append(d)
        return result


def update_host_seen(host_id: int):
    with get_conn() as c:
        c.execute("UPDATE hosts SET last_seen = ? WHERE id = ?",
                  (datetime.utcnow().isoformat(timespec="seconds"), host_id))


def delete_host(host_id: int):
    with get_conn() as c:
        c.execute("DELETE FROM hosts WHERE id = ?", (host_id,))


def update_host_config(host_id: int, polling_interval: int = None,
                       active: int = None, description: str = None):
    sets, params = [], []
    if polling_interval is not None:
        sets.append("polling_interval = ?"); params.append(int(polling_interval))
    if active is not None:
        sets.append("active = ?"); params.append(int(bool(active)))
    if description is not None:
        sets.append("description = ?"); params.append(description)
    if not sets:
        return
    params.append(host_id)
    with get_conn() as c:
        c.execute(f"UPDATE hosts SET {', '.join(sets)} WHERE id = ?", params)


# ---------- metrics ----------
```

```
def insert_metric(host_id: int, m: dict):
    with get_conn() as c:
        c.execute(
            """INSERT INTO metrics (host_id, ts, cpu, ram, disk, net_sent,
net_recv, processes, uptime)
               VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)""",
            (
                host_id,
                datetime.utcnow().isoformat(timespec="seconds"),
                float(m.get("cpu", 0)),
                float(m.get("ram", 0)),
                float(m.get("disk", 0)),
                float(m.get("net_sent", 0)),
                float(m.get("net_recv", 0)),
                int(m.get("processes", 0)),
                int(m.get("uptime", 0)),
            ),
        )


def get_metrics(host_id: int, since: datetime = None, limit: int = 500) -> list:
    """Return metrics for a host, oldest first, optionally since a given time."""
    with get_conn() as c:
        if since is None:
            rows = c.execute(
                """SELECT * FROM metrics WHERE host_id = ?
                   ORDER BY ts DESC LIMIT ?""",
                (host_id, limit),
            ).fetchall()
            rows = list(reversed(rows))
        else:
            rows = c.execute(
                """SELECT * FROM metrics WHERE host_id = ? AND ts >= ?
                   ORDER BY ts ASC LIMIT ?""",
                (host_id, since.isoformat(timespec="seconds"), limit),
            ).fetchall()
        return [dict(r) for r in rows]


def prune_old_metrics(days: int = 30):
    """Delete metrics older than N days. Call periodically."""
    cutoff = (datetime.utcnow() -
timedelta(days=days)).isoformat(timespec="seconds")
    with get_conn() as c:
        c.execute("DELETE FROM metrics WHERE ts < ?", (cutoff,))


# ---------- thresholds ----------

def get_thresholds(host_id: int = None) -> dict:
    """Return effective thresholds for a host, falling back to global ones."""
    with get_conn() as c:
        rows = c.execute(
            "SELECT * FROM thresholds WHERE host_id IS NULL OR host_id = ?",
            (host_id,),
```

```
        ).fetchall()
        out = {}
        for r in rows:
            # per-host overrides global
            if r["metric"] not in out or r["host_id"] is not None:
                out[r["metric"]] = {"warning": r["warning"], "critical":
r["critical"]}
        return out


def set_threshold(metric: str, warning: float, critical: float, host_id: int =
None):
    """Insert or update a threshold (global when host_id is None)."""
    with get_conn() as c:
        if host_id is None:
            row = c.execute(
                "SELECT id FROM thresholds WHERE host_id IS NULL AND metric = ?",
                (metric,),
            ).fetchone()
        else:
            row = c.execute(
                "SELECT id FROM thresholds WHERE host_id = ? AND metric = ?",
                (host_id, metric),
            ).fetchone()
        if row:
            c.execute(
                "UPDATE thresholds SET warning = ?, critical = ? WHERE id = ?",
                (float(warning), float(critical), row["id"]),
            )
        else:
            c.execute(
                """INSERT INTO thresholds (host_id, metric, warning, critical)
                   VALUES (?, ?, ?, ?)""",
                (host_id, metric, float(warning), float(critical)),
            )


# ---------- alerts ----------

def insert_alert(host_id: int, metric: str, level: str, message: str,
                 value: float = None, threshold: float = None):
    """Insert an alert, suppressing identical unresolved alerts within the last 5
minutes."""
    now = datetime.utcnow()
    cutoff = (now - timedelta(minutes=5)).isoformat(timespec="seconds")
    with get_conn() as c:
        recent = c.execute(
            """SELECT id FROM alerts
               WHERE host_id = ? AND metric = ? AND level = ?
                 AND resolved = 0 AND ts >= ?""",
            (host_id, metric, level, cutoff),
        ).fetchone()
        if recent:
            return
        c.execute(
```

```
            """INSERT INTO alerts (host_id, ts, metric, level, value, threshold,
message)
               VALUES (?, ?, ?, ?, ?, ?, ?)""",
            (host_id, now.isoformat(timespec="seconds"),
             metric, level, value, threshold, message),
        )


def list_alerts(host_id: int = None, only_unresolved: bool = False, limit: int =
200) -> list:
    sql = """SELECT a.*, h.hostname FROM alerts a
             JOIN hosts h ON h.id = a.host_id"""
    where, params = [], []
    if host_id is not None:
        where.append("a.host_id = ?"); params.append(host_id)
    if only_unresolved:
        where.append("a.resolved = 0")
    if where:
        sql += " WHERE " + " AND ".join(where)
    sql += " ORDER BY a.ts DESC LIMIT ?"
    params.append(limit)
    with get_conn() as c:
        return [dict(r) for r in c.execute(sql, params).fetchall()]


def resolve_alert(alert_id: int):
    with get_conn() as c:
        c.execute("UPDATE alerts SET resolved = 1 WHERE id = ?", (alert_id,))


# ---------- commands (remote process management) ----------

def queue_command(host_id: int, cmd_type: str, payload: dict, issued_by: str) ->
int:
    """Queue a new command for the host. The agent picks it up on its next
poll."""
    if cmd_type not in ("list_processes", "kill_process", "exec"):
        raise ValueError(f"unknown command type: {cmd_type}")
    with get_conn() as c:
        cur = c.execute(
            """INSERT INTO commands (host_id, type, payload, status, created_at,
issued_by)
               VALUES (?, ?, ?, 'pending', ?, ?)""",
            (host_id, cmd_type, _json.dumps(payload),
             datetime.utcnow().isoformat(timespec="seconds"), issued_by),
        )
        return cur.lastrowid


def get_pending_commands(host_id: int, mark_executing: bool = True) -> list:
    """Return pending commands for this host. By default also marks them as
    'executing' so the same command is not handed out twice."""
    with get_conn() as c:
        rows = c.execute(
            """SELECT * FROM commands WHERE host_id = ? AND status = 'pending'
               ORDER BY created_at ASC LIMIT 20""",
```

```
            (host_id,),
        ).fetchall()
        out = []
        for r in rows:
            d = dict(r)
            try:
                d["payload"] = _json.loads(d["payload"])
            except Exception:
                d["payload"] = {}
            out.append(d)
        if mark_executing and out:
            ids = [r["id"] for r in out]
            qs = ",".join("?" * len(ids))
            c.execute(
                f"UPDATE commands SET status = 'executing' WHERE id IN ({qs})",
                ids,
            )
        return out


def save_command_result(cmd_id: int, status: str, exit_code: int = None,
                        result: dict = None, error: str = None):
    """Called by the agent after executing a command."""
    if status not in ("done", "failed"):
        raise ValueError(f"invalid status: {status}")
    with get_conn() as c:
        c.execute(
            """UPDATE commands
               SET status = ?, executed_at = ?, exit_code = ?, result = ?, error
= ?
               WHERE id = ?""",
            (status,
             datetime.utcnow().isoformat(timespec="seconds"),
             exit_code,
             _json.dumps(result) if result is not None else None,
             error,
             cmd_id),
        )


def get_command(cmd_id: int):
    with get_conn() as c:
        row = c.execute("SELECT * FROM commands WHERE id = ?",
(cmd_id,)).fetchone()
        if not row:
            return None
        d = dict(row)
        try:
            d["payload"] = _json.loads(d["payload"]) if d["payload"] else {}
        except Exception:
            d["payload"] = {}
        try:
            d["result"] = _json.loads(d["result"]) if d["result"] else None
        except Exception:
            d["result"] = None
        return d
```

```
def list_commands(host_id: int, limit: int = 30, types: list = None) -> list:
    """Return recent commands for a host, newest first."""
    sql = "SELECT * FROM commands WHERE host_id = ?"
    params = [host_id]
    if types:
        sql += f" AND type IN ({','.join('?'*len(types))})"
        params.extend(types)
    sql += " ORDER BY created_at DESC LIMIT ?"
    params.append(limit)
    with get_conn() as c:
        rows = c.execute(sql, params).fetchall()
        out = []
        for r in rows:
            d = dict(r)
            try: d["payload"] = _json.loads(d["payload"]) if d["payload"] else {}
            except Exception: d["payload"] = {}
            try: d["result"] = _json.loads(d["result"]) if d["result"] else None
            except Exception: d["result"] = None
            out.append(d)
        return out


def get_latest_process_list(host_id: int):
    """Return the most recent successful list_processes result for this host, or
None."""
    with get_conn() as c:
        row = c.execute(
            """SELECT * FROM commands
               WHERE host_id = ? AND type = 'list_processes' AND status = 'done'
               ORDER BY executed_at DESC LIMIT 1""",
            (host_id,),
        ).fetchone()
        if not row:
            return None
        d = dict(row)
        try:
            d["result"] = _json.loads(d["result"]) if d["result"] else None
        except Exception:
            d["result"] = None
        return d


def cleanup_stuck_commands(stale_minutes: int = 5):
    """Move commands stuck in the 'executing' state back to failed (agent went
away)."""
    cutoff = (datetime.utcnow() -
timedelta(minutes=stale_minutes)).isoformat(timespec="seconds")
    with get_conn() as c:
        c.execute(
            """UPDATE commands
               SET status = 'failed', error = 'timeout: agent did not return a
result',
                   executed_at = ?
               WHERE status = 'executing' AND created_at < ?""",
```

```
            (datetime.utcnow().isoformat(timespec="seconds"), cutoff),
        )
```

Prilog 5: Poslužiteljska aplikacija – alerts.py

```
"""
alerts.py
---------
Infrastructure for the alerting subsystem: the background thread that
periodically scans for hosts that have gone offline.

The alerting policy itself (what counts as a warning, a critical, or an offline
host) lives in services.AlertService. This module only owns the threading
concern - it runs that policy on a timer.
"""

import threading
import time

from services import AlertService


def offline_monitor_loop(interval_s: int = 30):
    while True:
        try:
            AlertService.check_offline_hosts()
        except Exception as e:
            print(f"[offline_monitor] error: {e}")
        time.sleep(interval_s)


def start_background_monitor(interval_s: int = 30):
    t = threading.Thread(target=offline_monitor_loop,
                         args=(interval_s,), daemon=True)
    t.start()
    return t
```

Prilog 6: Skup automatiziranih provjera – tests.py

```
import os
import sys
import tempfile

import database as db
db.DB_PATH = os.path.join(tempfile.mkdtemp(), "test.db")

import server
from services import (HostService, MetricsService, ThresholdService,
                      AlertService, CommandService, ValidationError)

_failures = 0
```

```
def check(condition, message):
    global _failures
    if condition:
        print("PASS - " + message)
    else:
        _failures += 1
        print("FAIL - " + message)


def main():
    db.init_db()
    server.app.config["TESTING"] = True
    client = server.app.test_client()

    r = client.post("/api/register", json={"hostname": "vm-test", "description":
"ci"})
    check(r.status_code == 201, "agent registration returns 201")
    key = r.get_json()["api_key"]
    host_id = r.get_json()["host_id"]

    r = client.post("/api/metrics", headers={"X-API-Key": key},
                    json={"cpu": 150, "ram": -5, "disk": 99,
                          "net_sent": -10, "processes": 200, "uptime": 3600})
    check(r.status_code == 200, "metric submission accepted")

    latest = MetricsService.history(host_id, "1h")[-1]
    check(latest["cpu"] == 100.0, "cpu of 150 is clamped to 100")
    check(latest["ram"] == 0.0, "ram of -5 is clamped to 0")
    check(latest["net_sent"] == 0.0, "negative net_sent is floored to 0")

    check(any(a["metric"] == "cpu" and a["level"] == "critical"
              for a in AlertService.list(host_id=host_id)),
          "critical CPU alert raised when value crosses the limit")

    check(client.post("/api/metrics", headers={"X-API-Key": "bogus"},
                      json={}).status_code == 401,
          "invalid API key is rejected with 401")

    try:
        ThresholdService.save("cpu", warning=90, critical=80)
        check(False, "warning >= critical must be rejected")
    except ValidationError:
        check(True, "warning >= critical is rejected")

    try:
        CommandService.request_kill(host_id, pid=1, signal="TERM", issued_by="t")
        check(False, "killing PID 1 must be rejected")
    except ValidationError:
        check(True, "killing PID <= 1 is rejected")

    try:
        CommandService.request_exec(host_id, cmd="  ", timeout=30, issued_by="t")
        check(False, "empty command must be rejected")
    except ValidationError:
        check(True, "empty exec command is rejected")
```

```
    cid = CommandService.request_exec(host_id, cmd="uname -a", timeout=99999,
issued_by="t")
    check(db.get_command(cid)["payload"]["timeout"] == 600,
          "oversized exec timeout is clamped to 600")

    CommandService.request_process_list(host_id, "t")
    check(CommandService.request_process_list(host_id, "t") is None,
          "duplicate refresh request is debounced")

    handed = client.post("/api/metrics", headers={"X-API-Key": key},
                         json={"cpu": 1, "ram": 1, "disk": 1}).get_json()
    check(len(handed["commands"]) >= 1, "pending commands are handed to the
agent")

    host = next(h for h in HostService.list_with_status() if h["id"] == host_id)
    check(host["online"] is True, "a freshly seen host reports as online")

    print()
    if _failures:
        print(f"{_failures} check(s) failed.")
        sys.exit(1)
    print("All checks passed.")


if __name__ == "__main__":
    main()
```